

CS473 -Deep Learning

Module 1

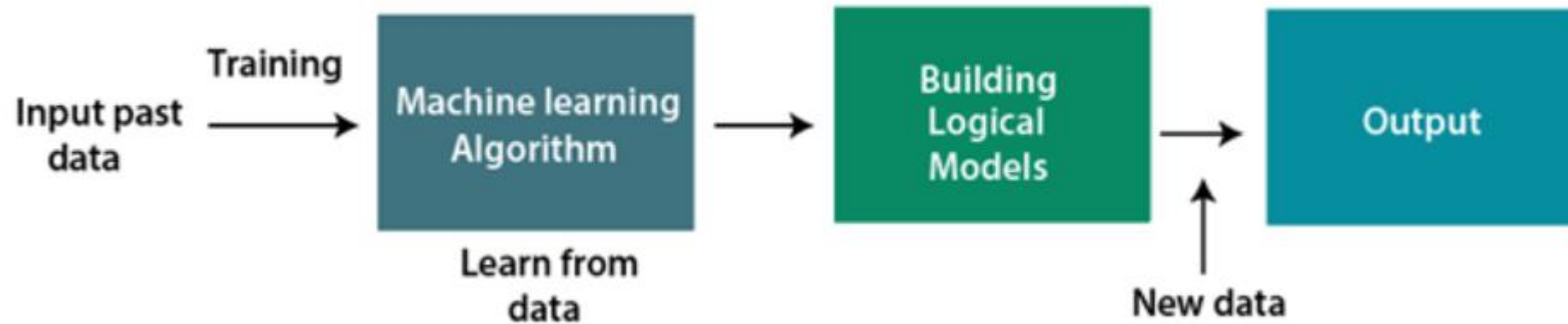
Module 1

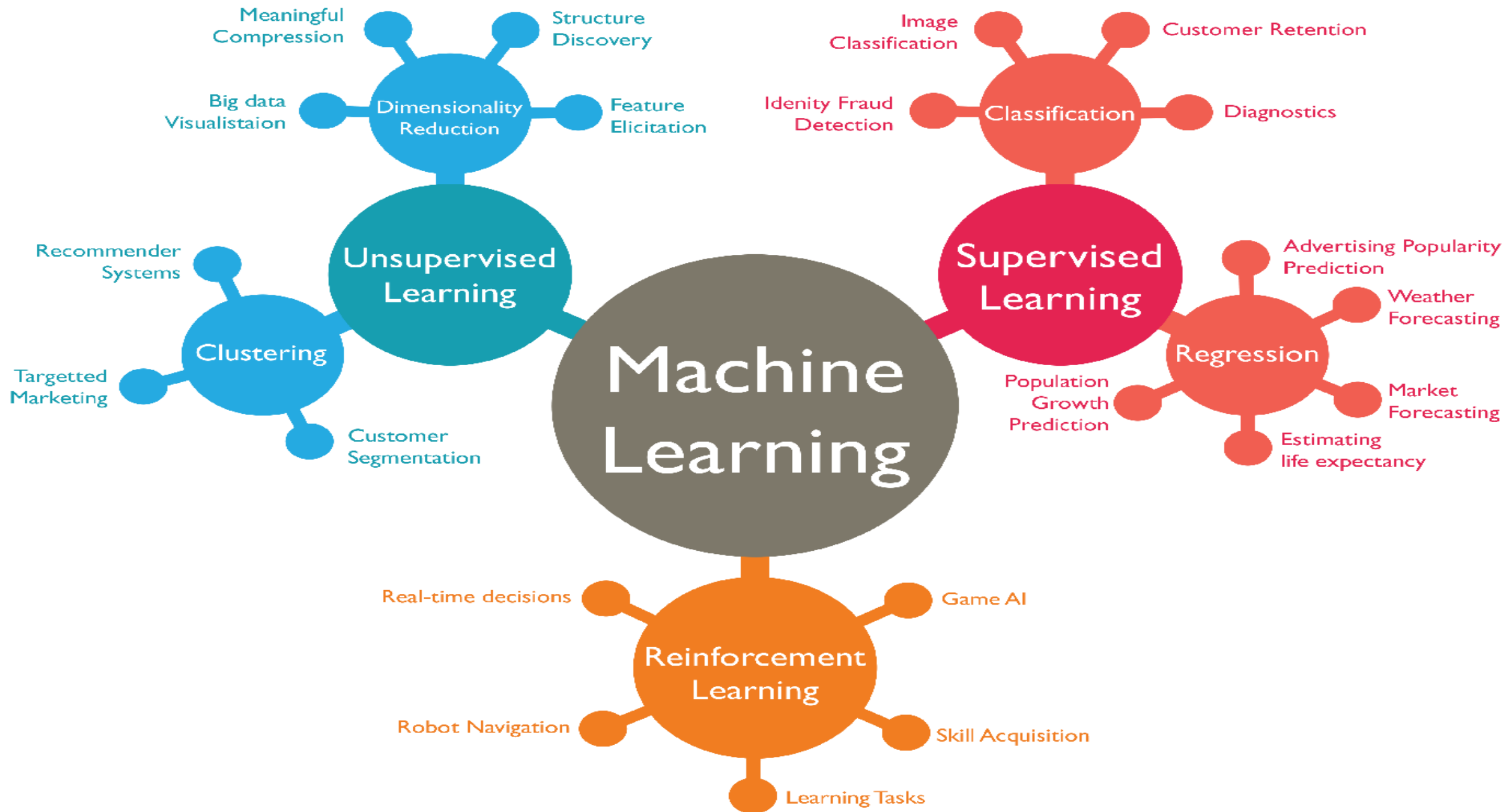
- Machine Learning Basics: Learning Algorithms, Capacity, Over Fitting and Under fitting, Hyperparameters and Validation sets, Estimators, Bias and variance, Maximum Likelihood Estimation, BayesianStatistics, Supervised and Unsupervised Learning algorithms, SGD, Building a ML algorithm,

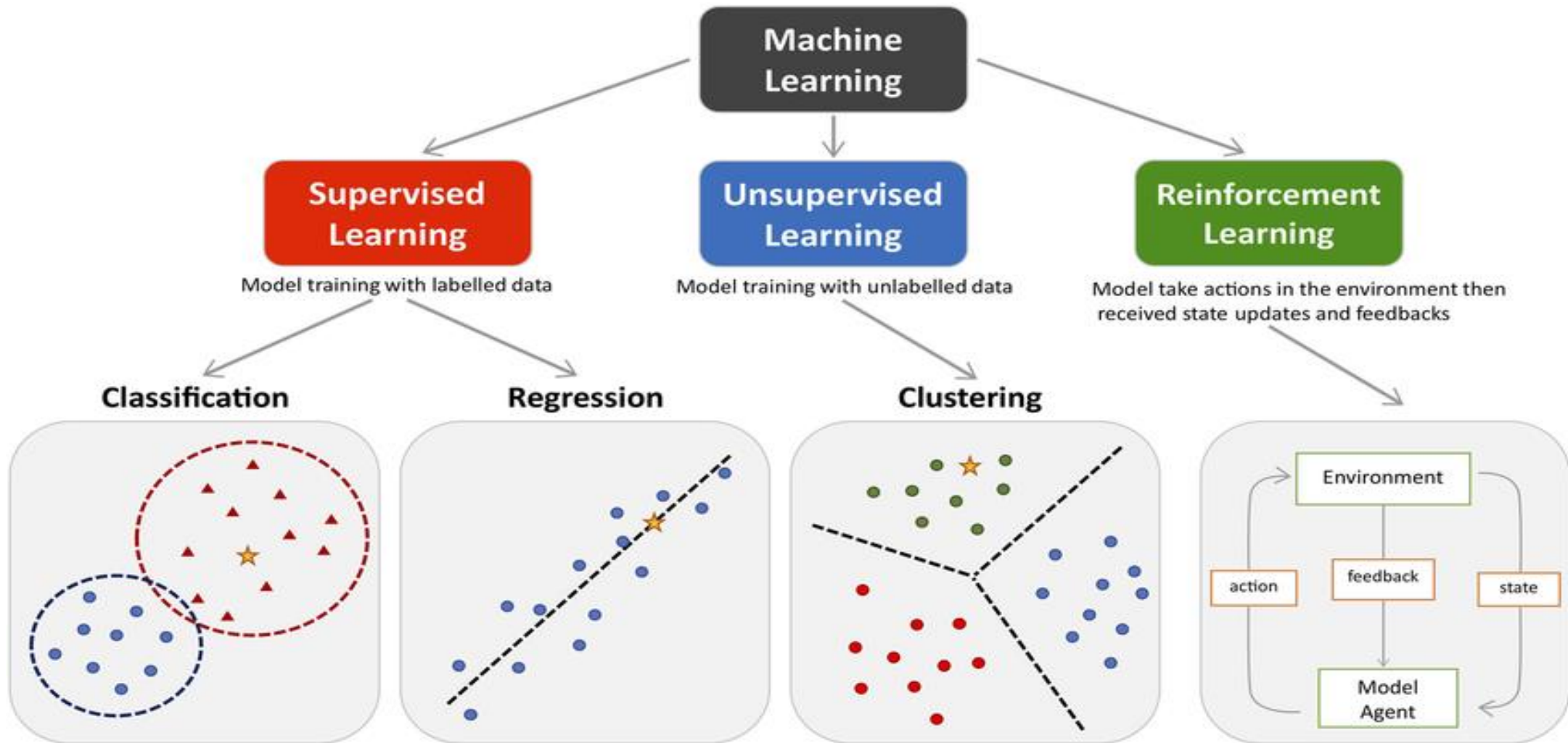
Machine Learning Basics

- A machine learning algorithm is an algorithm that is able to learn from data.
- what do we mean by learning?
- Mitchell (1997) provides the definition
- “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .”

Machine Learning Basics







Machine Learning Basics

- Machine learning allows us to tackle tasks that are too difficult to solve with fixed programs written and designed by human beings.
- Learning is our means of attaining the ability to perform the task.
- For example, if we want a robot to be able to walk, then walking is the task.
- We could program the robot to learn to walk, or we could attempt to directly write a program that specifies how to walk manually.

Machine Learning Basics

- An example is a collection of features that have been quantitatively measured from some object or event that we want the machine learning system to process.
- We typically represent an example as a vector $x \in \mathbb{R}^n$ where each entry x_i of the vector is another feature.
- For example, the features of an image are usually the values of the pixels in the image.

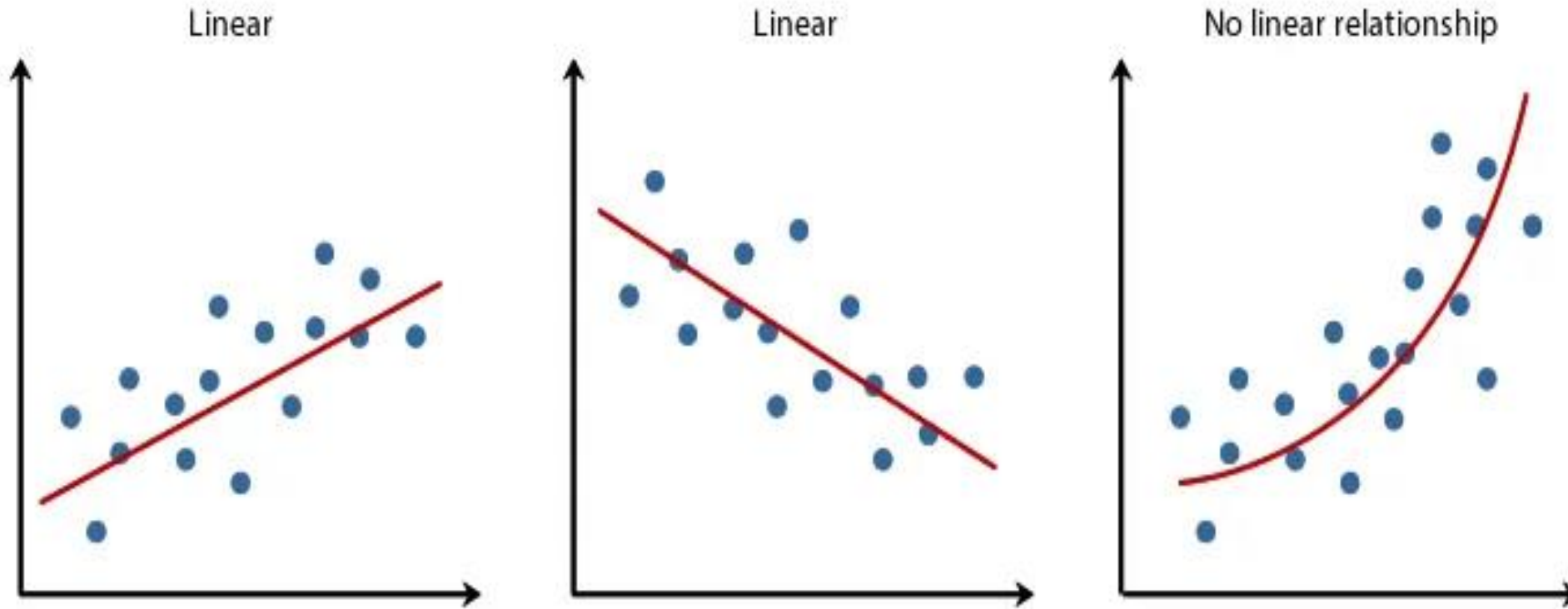
Machine Learning Basics

- Many kinds of tasks can be solved with machine learning.
- **Classification:**
- In this type of task, the computer program is asked to specify which of k categories some input belongs to.
- To solve this task, the learning algorithm is usually asked to produce a function $f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$.
- When $y = f(x)$, the model assigns an input described by vector x to a category identified by numeric code y .
- There are other variants of the classification task, for example, where f outputs a probability distribution over classes.

Machine Learning Basics

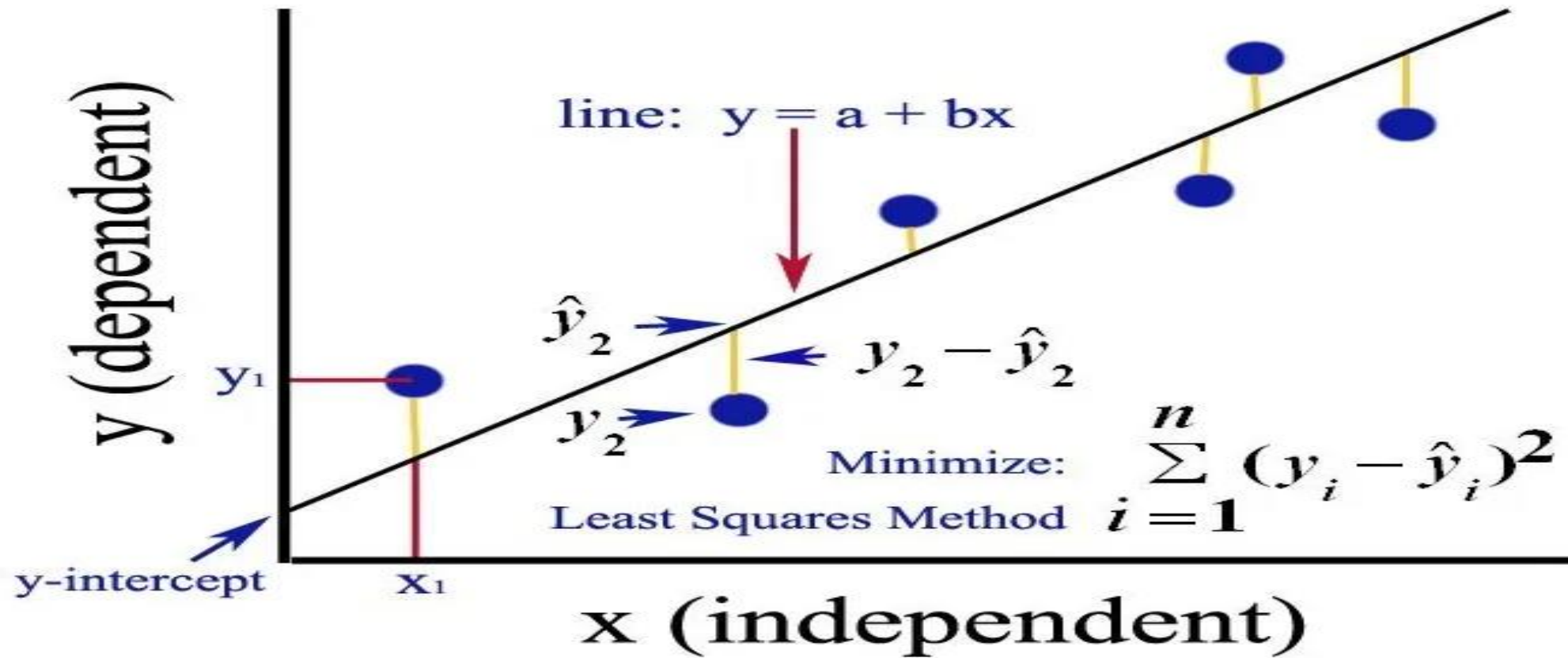
- **Classification:**
- An example of a classification task is object recognition, where the input is an image (usually described as a set of pixel brightness values), and the output is a numeric code identifying the object in the image.
- **Regression:** In this type of task, the computer program is asked to predict a numerical value given some input.
- To solve this task, the learning algorithm is asked to output a **function $f : \mathbf{R}^n \rightarrow \mathbf{R}$** . This type of task is similar to classification, except that the format of output is different.
- An example of a regression task is the prediction of the expected claim amount that an insured person will make (used to set insurance premiums),
- or the prediction of future prices of securities

Machine Learning Basics



<https://medium.datadriveninvestor.com/linear-regression-and-its-mathematical-implementation-29d520a75ede>

Machine Learning Basics



Machine Learning Basics

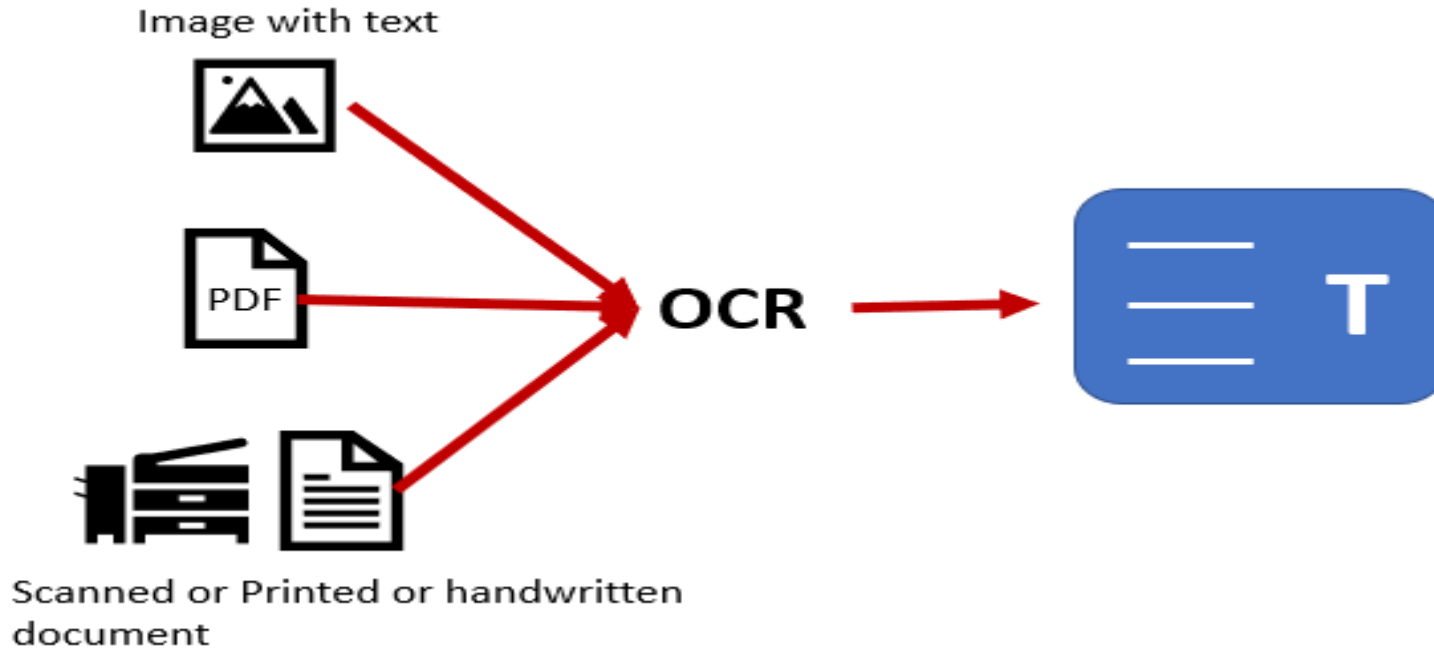
Transcription: In this type of task, the machine learning system is asked to observe a relatively unstructured representation of some kind of data and transcribe it into discrete, textual form.

For example, in **optical character recognition**, the computer program is shown a photograph containing an image of text and is asked to return this text in the form of a sequence of characters (e.g., in ASCII or Unicode format).

Another example is **speech recognition**, where the computer program is provided an audio waveform and emits a sequence of characters or word ID codes describing the words that were spoken in the audio recording.

Deep learning is a crucial component of modern speech recognition systems used at major companies including Microsoft, IBM and Google

Machine Learning Basics

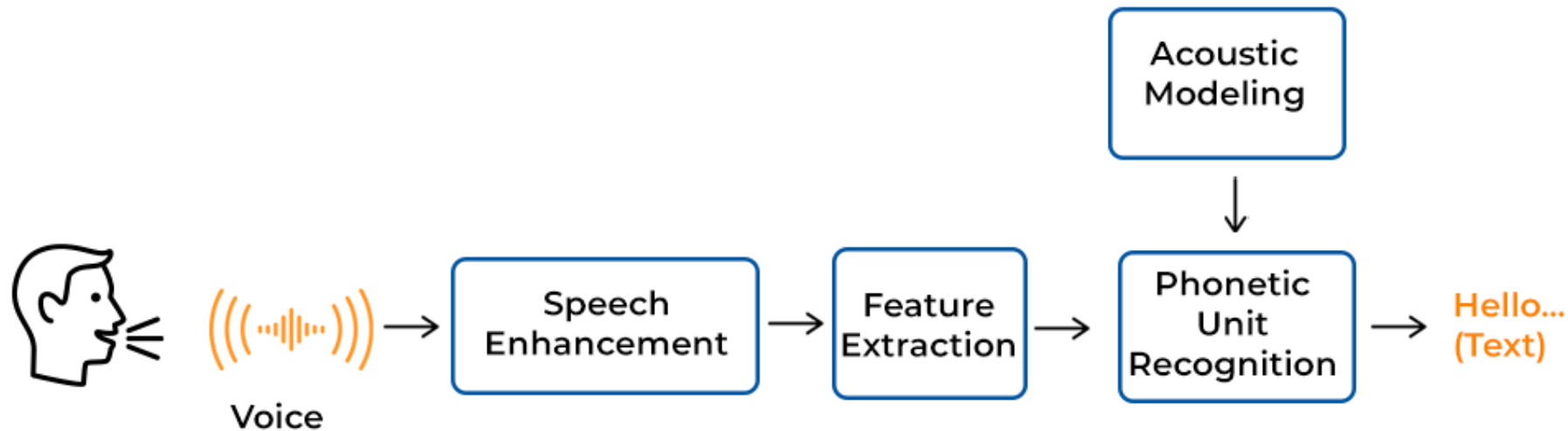


By transforming printed or handwritten text into data that computers can read, OCR is a cutting-edge technology that connects the physical and digital worlds

Machine Learning Basics



SPEECH RECOGNITION PROCESS



Machine Learning Basics

Machine translation:

This is commonly applied to natural languages, such as **translating from English to French**.

Structured output:

where the output is a vector with important relationships between the different elements.

One example is parsing—mapping a natural language sentence into a tree that describes its grammatical structure and tagging nodes of the trees as being verbs, nouns, or adverbs, and so on.

example, deep learning can be used to annotate the locations of roads in aerial photographs

For example, in image captioning, the computer program observes an image and outputs a natural language sentence describing the image .

These tasks are called structured output tasks because the program must output several values that are all tightly inter-related.

For example, the words produced by an image captioning program must form a valid sentence.

Machine Learning Basics

**a train traveling down a track
next to a forest.**



**a group of young boys playing
soccer on a field.**



Evergreen*

<https://evergreen.team/articles/automatic-image-captioning.html>

Machine Learning Basics

Anomaly detection:

An example of an anomaly detection task is credit card fraud detection.

By modeling your purchasing habits, a credit card company can detect misuse of your cards.

If a thief steals your credit card or credit card information, the thief's purchases will often come from a different probability distribution over purchase types than your own

Machine Learning Basics

Synthesis and sampling:

In this type of task, the machine learning algorithm is asked to generate new examples that are similar to those in the training data.

Synthesis and sampling via machine learning can be useful for media applications where it can be expensive or boring for an artist to generate large volumes of content by hand.

For example, video games can automatically generate textures for large objects or landscapes, rather than requiring an artist to manually label each pixel.

For example, in a speech synthesis task, we provide a written sentence and ask the program to emit an audio waveform containing a spoken version of that sentence.

Machine Learning Basics

Imputation of missing values:

The algorithm must provide a prediction of the values of the missing entries.

Denoising:

The learner must predict the clean example x from its corrupted version $\tilde{x}$, or more generally predict the conditional probability distribution $p(x \mid \tilde{x})$.

Machine Learning Basics

Density estimation or probability mass function estimation:

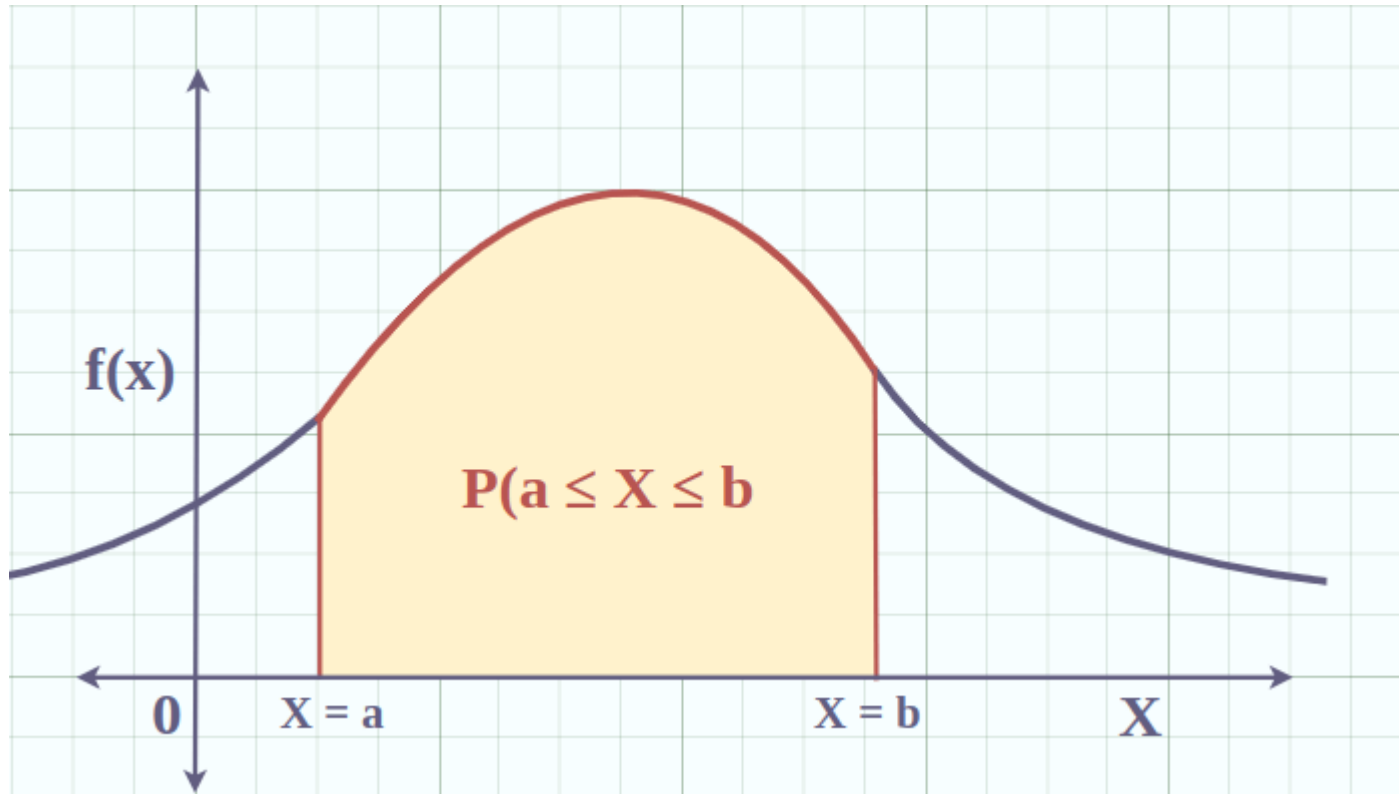
the machine learning algorithm is asked to learn a function $p_{\text{model}} : \mathbb{R}^n \rightarrow \mathbb{R}$, where $p_{\text{model}}(x)$ can be interpreted as a probability density function (if x is continuous)

or a probability mass function (if x is discrete) on the space that the examples were drawn from.

the algorithm needs to learn the structure of the data it has seen.

It must know where examples cluster tightly and where they are unlikely to occur.

Machine Learning Basics



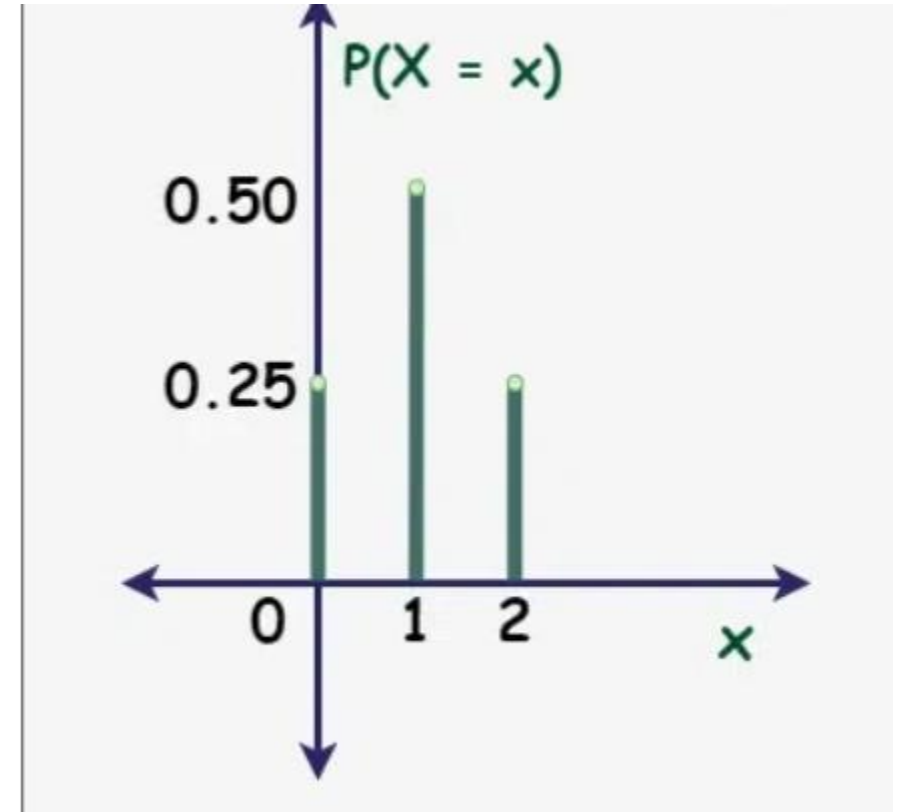
If X is continuous random variable and $f(x)$ be the probability density function

The following graph gives the probability for X lying between interval a and b .

Machine Learning Basics

x	$0\{HH\}$	$1\{HT, TH\}$	$2\{TT\}$
$P(X = x)$	$1/4$	$2/4 = 1/2$	$1/4$

Let a coin is tossed two times and X be the random variable representing the number of tails then, the probability mass function table for above event is given below



Probability-mass-function (PMF)

The Performance Measure

- In order to evaluate the abilities of a machine learning algorithm, we must design a quantitative measure of its performance.
- For tasks such as **classification**, classification with missing inputs, and transcription, we often measure the **accuracy of the model**.
- **Accuracy** is just the proportion of examples for which the model produces the correct output.
- We can also obtain equivalent **information by measuring the error rate**, the proportion of examples for which the model produces an incorrect output.
- We often refer to the error rate as the expected 0-1 loss.
- **The 0-1 loss on a particular**
 - example is 0 if it is correctly classified
 - and 1 if it is not.

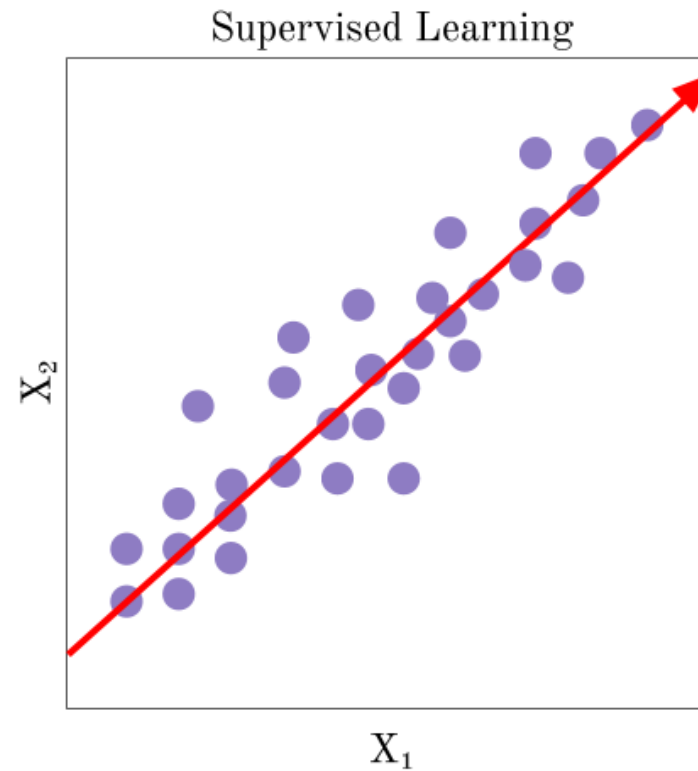
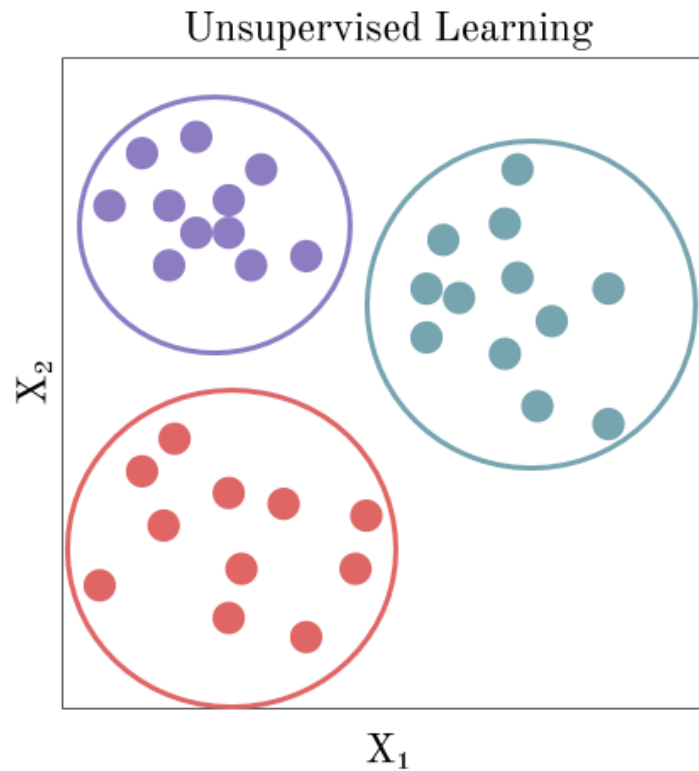
The Performance Measure, P

- For tasks such as **density estimation**, it does not make sense to measure accuracy, error rate, or any other kind of 0-1 loss.
- Instead, we must use a different performance metric that gives the model a continuous-valued score for each example.
- The most common approach is to report the **average log-probability** the model assigns to some examples.
- we are interested in how well the machine learning algorithm performs on data that it has not seen before,
- since this determines how well it will work when deployed in the real world.
- We therefore evaluate these performance measures using a **test set** of data that is separate from the data used for **training** the machine learning system.

The Experience, E

- Machine learning algorithms can be broadly categorized **as unsupervised or supervised** by what kind of experience they are allowed to have during the learning process.
- A dataset is a collection of many data points.
- **Unsupervised learning** algorithms experience a dataset containing many features, then learn useful properties of the structure of this dataset.
- In the context of deep learning, we usually want to learn the entire probability distribution that generated a dataset, whether explicitly as in density estimation or implicitly for tasks like synthesis or denoising.
- **clustering**, which consists of dividing the dataset into clusters of similar examples.

The Experience, E



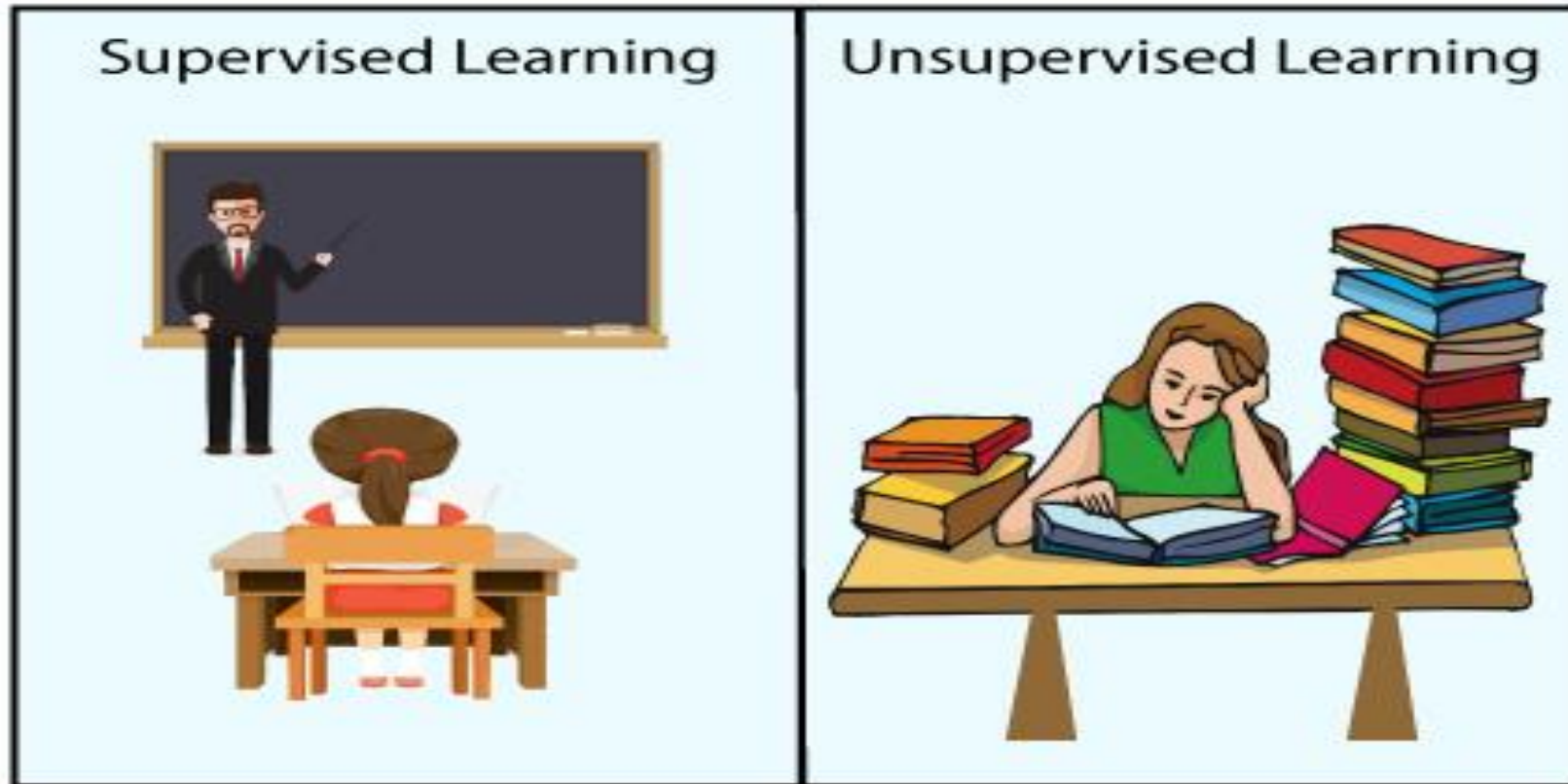
The Experience, E

- **Supervised learning** algorithms experience a dataset containing features, but each example is also associated with a **label or target**. For example, the Iris dataset is annotated with the species of each iris plant.
- **A supervised learning** algorithm can study the Iris dataset and learn to classify iris plants into three different species based on their measurements.
- **unsupervised learning** involves observing several examples of a random vector x , and attempting to implicitly or explicitly learn the **probability distribution $p(x)$** , of that distribution.

The Experience, E

- **supervised learning** involves observing several examples of a random vector x and an associated value or vector y , and learning to predict y from x , usually by estimating $p(y \mid x)$.
- The term supervised learning originates from the view of the target y being provided by an **instructor or teacher** who shows the machine learning system what to do.
- In **unsupervised learning**, there is no **instructor or teacher**, and the algorithm must learn to make sense of the data without this guide

The Experience, E



The Experience, E

- Many machine learning technologies can be used to perform both tasks.
- For example, the chain rule of probability states that for a vector $x \in \mathbb{R}^n$, the joint distribution can be decomposed as
- $P(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$
- we can solve the ostensibly **unsupervised problem** of modeling $p(x)$ by splitting it into **n supervised learning problems**.
- We can solve the **supervised learning problem** of learning $p(y | x)$ by using traditional unsupervised learning technologies to learn the joint distribution $p(x, y)$ and inferring
- $p(y | x) = \frac{p(x, y)}{\sum_{y'} p(x, y')}$

The Experience, E

- Traditionally, people refer to **regression, classification and structured output problems** as supervised learning.
- **Density estimation in support** of other tasks is usually considered unsupervised learning.
- **semisupervised learning**, some examples include a supervision target but others do not.
- In multi-instance learning, an entire collection of examples is labeled as containing or not containing an example of a class, but the individual members of the collection are not labeled

The Experience, E

- Some machine learning algorithms do not just experience a fixed dataset.
- For example, **reinforcement learning algorithms** interact with an environment, so there is a feedback loop between the learning system and its experiences.
- a dataset is a collection of examples, which are in turn collections of features.
- One common way of describing a dataset is with a design matrix.
- For instance, the Iris dataset contains 150 examples with four features for each example.
- This means we can represent the dataset with a design matrix $X \in \mathbf{R}^{150 \times 4}$
- , where $X_{i,1}$ is the sepal length of plant i ,
- $X_{i,2}$ is the sepal width of plant i , etc.

Linear Regression

- linear regression solves a regression problem.
- the goal is to build a system that can take a vector $\mathbf{x} \in \mathbb{R}^n$ as input and predict the value of a scalar $y \in \mathbb{R}$ as its output.
- In the case of linear regression, the output is a linear function of the input.
- Let $\hat{y}$ be the value that our model predicts y should take on.
- We define the output to be $\hat{\mathbf{y}} = \mathbf{w}^t \mathbf{x}$
- where $\mathbf{w} \in \mathbb{R}^n$ is a vector of parameters

Linear Regression

- We define the output to be $\hat{y} = \mathbf{w}^t \mathbf{x}$
- Parameters are values that control the behavior of the system.
- w_i is the coefficient that we multiply by feature x_i before summing up the contributions from all the features.
- We can think of $\mathbf{w}$ as a set of weights that determine how each feature affects the prediction.
- If a feature x_i receives a positive weight w_i , then increasing the value of that feature increases the value of our prediction $\hat{y}$.
- If a feature receives a negative weight, then increasing the value of that feature decreases the value of our prediction.
- If a feature's weight is large in magnitude, then it has a large effect on the prediction.
- If a feature's weight is zero, it has no effect on the prediction.

Linear Regression

- We thus have a definition of our task T :
- to predict y from x by outputting $\hat{y} = \mathbf{w}^t \mathbf{x}$
- Next we need a definition of our performance measure, P .
- Suppose that we have a design matrix of m example inputs that we will not use for training, only for evaluating how well the model performs.
- We also have a vector of regression targets providing the correct value of y for each of these examples.
- Because this dataset will only be used for evaluation, we call it the **test set**. We refer to the design matrix of inputs as $\mathbf{X}^{(\text{test})}$
- and the vector of regression targets as $\mathbf{y}^{(\text{test})}$

Linear Regression

- One way of measuring the performance of the model is to compute the mean squared error of the model on the test set.
- If $\hat{y}^{(test)}$ gives the predictions of the model on the test set, then the mean squared error is given by
- $$\mathbf{MSE}_{test} = \frac{1}{m} \sum_i (\hat{y}^{(test)} - y^{(test)})^2_i$$
- Intuitively, one can see that this error measure decreases to 0
- when $\hat{y}^{(test)} = y^{(test)}$
- We can also see that
- $$\mathbf{MSE}_{test} = \frac{1}{m} \sum_i \|\hat{y}^{(test)} - y^{(test)}\|_2^2$$
- so the error increases whenever the Euclidean distance between the predictions and the targets increases.

Linear Regression

- we need to design an algorithm that will **improve the weights $\mathbf{w}$** in a way that **reduces MSE_{test}** when the algorithm is allowed to gain experience by observing a **training set** ($X(\text{train})$, $y(\text{train})$).
- Solution is just to minimize the **mean squared error** on the training set, **MSE_{train}** .
- To **minimize MSE_{train}** , we can simply solve for where **its gradient is 0**:
- $\nabla_{\mathbf{w}} MSE_{train} = \mathbf{0}$
- $\nabla_{\mathbf{w}} \frac{1}{m} \sum_i \|\hat{\mathbf{y}}^{(train)} - \mathbf{y}^{(train)}\|_2^2 = \mathbf{0}$
- $\frac{1}{m} \nabla_{\mathbf{w}} \sum_i \|\mathbf{X}^{(train)} \mathbf{w} - \mathbf{y}^{(train)}\|_2^2 = \mathbf{0}$

Linear Regression

$$\nabla_w \left(X^{(train)} w - y^{(train)} \right)^T \left(X^{(train)} w - y^{(train)} \right) = 0$$

$$\nabla_w \left(w^T X^{(train)T} X^{(train)} w - 2w^T X^{(train)T} y^{(train)} \right) + y^{(train)T} y^{(train)} = 0$$

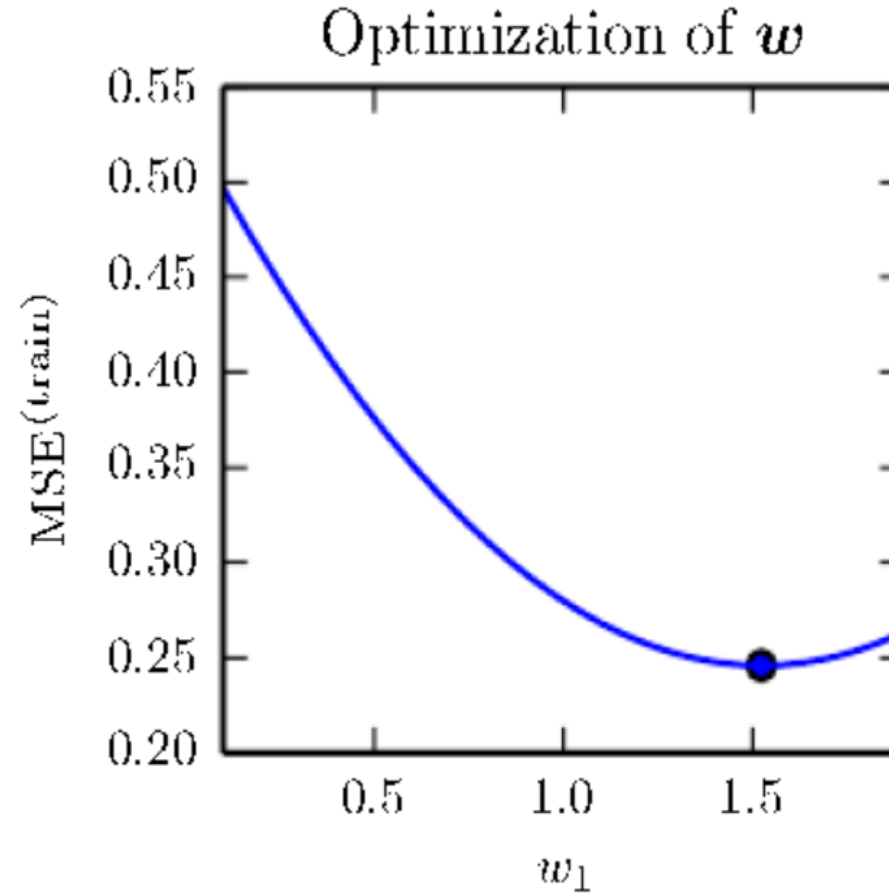
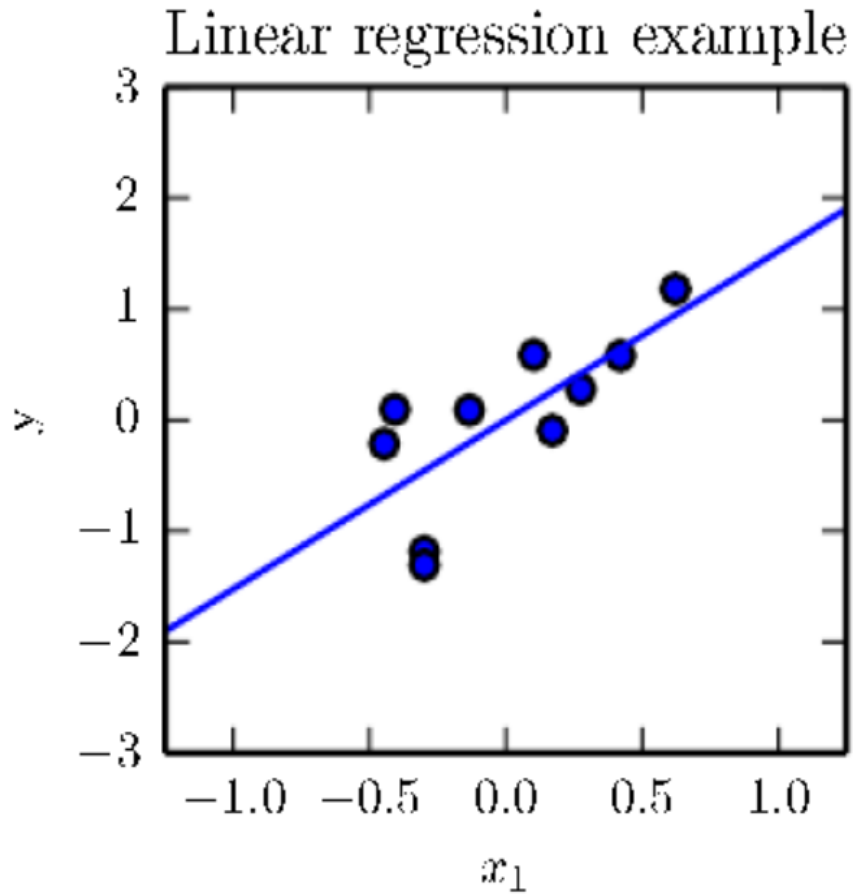
∇_w = Partial derivative w.r.t w

So ,

- $2X^{(train)T} X^{(train)} w - 2X^{(train)T} y^{(train)} = 0$

- $w = \left(X^{(train)T} X^{(train)} \right)^{-1} X^{(train)T} y^{(train)}$

normal equations



A linear regression problem, with a training set consisting of ten data points, each containing one feature. Because there is only one feature, the weight vector w contains only a single parameter to learn, w_1 .

(Left) Observe that linear regression learns to set w_1 such that the line $y = w_1x$ comes as close as possible to passing through all the training points.

(Right) The plotted point indicates the value of w_1 found by the normal equations, which we can see minimizes the mean squared error on the training set.

Linear Regression

- It is worth noting that the term linear regression is often used to refer to
- a slightly more sophisticated model with one additional parameter—an **intercept term b** .
- In this model,
- $\hat{y} = \mathbf{w}^T \mathbf{x} + b$
- so the mapping from parameters to predictions is still a linear function but the
- mapping from features to predictions is now an affine function.
- This extension to affine functions means that the plot of the model's predictions still looks like a line, but it need not pass through the origin

Linear Regression

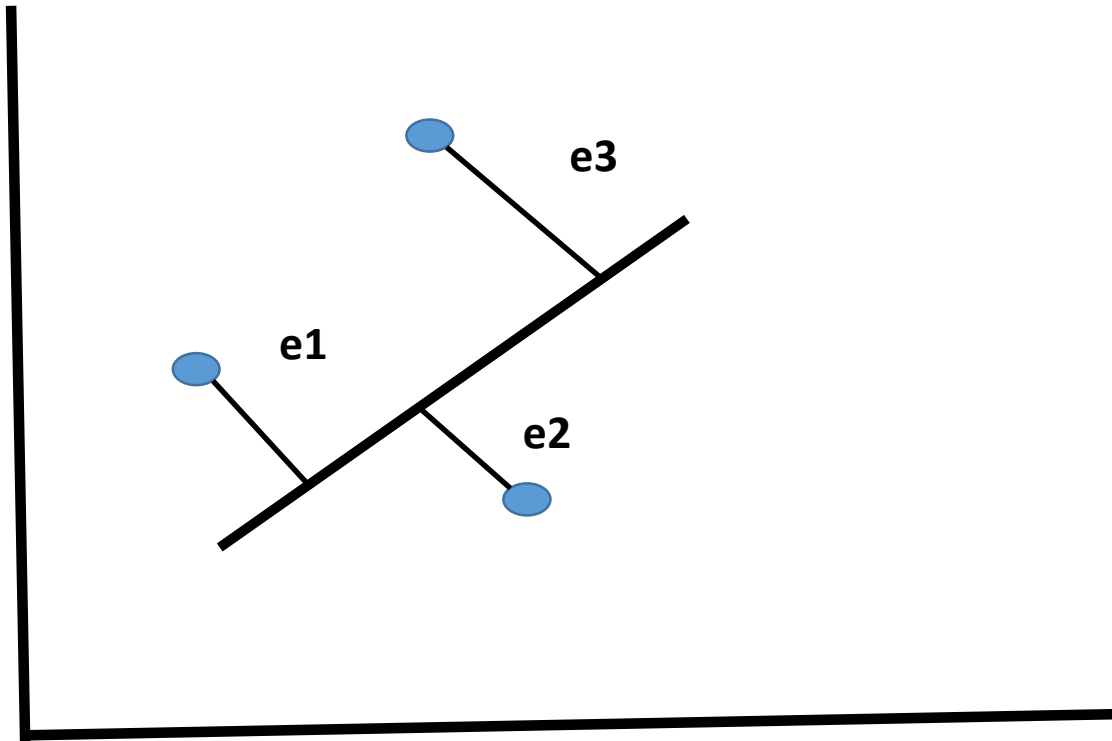
Linear Regression Example

- $y = a_0 + a_1x + \epsilon$
- a_0 intercept (Bias)
- a_1 Slope
- ϵ error in the prediction.

Assumption of Linear Regression

- 1) y is random & mutually independent
- 2) $\epsilon = |y_i - y_o|$
- y_i = Predicted output
- y_o = True output

Linear Regression Example



- $y_1 = (a_0 + a_1 x_1) + \epsilon_1$

- $y_2 = (a_0 + a_2 x_2) + \epsilon_2$

- .

- $y_n = (a_0 + a_n x_n) + \epsilon_n$

- $\epsilon_i = y_i - (a_0 + a_i x_i)$

- $\langle e_1, e_2, \dots, e_n \rangle$ are errors detailed

- $E = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - (a_0 + a_i x_i))$

- As the minimisation of sum of absolute values of individual errors
- $E = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - (a_0 + a_i x_i))$
- $E = \sum_{i=1}^n |e_i| = \sum_{i=1}^n |(y_i - (a_0 + a_i x_i))|$
- **Or sum of square**
- $E = \sum_{i=1}^n (e_i^2) = \sum_{i=1}^n ((y_i - (a_0 + a_i x_i))^2$
- So, Linear regression is modelled as a minimization function as follows
- $J(a, b) = \sum_{i=1}^n ((y_i - f(x_i))^2$
- $F(x_i) = (a_0 + a_i x_i)$
- Criteria function of parameter a_0 & a_i . These need to be minimized.
- Solution
- **Differentiate and zero**

- Differentiate and zero

- $a_1 = \frac{(\overline{xy}) - \bar{x} \cdot \bar{y}}{(\overline{x_i^2}) - (\bar{x})^2}$

- $a_0 = \bar{y} - a_1 \bar{x}$

Xi (Week)	Yi (Solution)	$\left((x_i)^2 \right)$	$x_i \times y_i$
1	1.2	1	1.2
2	1.8	4	3.6
3	2.6	9	7.8
4	3.2	16	12.8
5	3.8	25	19
Sum=15 $\bar{x} = 15/5$ =3	Sum=12.6 $\bar{y} = 12.6/5$ =2.52	Sum=55 $\overline{(x_i)^2} = 55/5=11$	Sum= 44.4 $\bar{x} \bar{y} = 44.5/5$ 8.88

Xi (Week)	Yi (Solution)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

$$a_1 = \frac{(8.88 - 3 * 2.52)}{11 - (3)^2} = 0.66$$

$$a_0 = 2.52 - 0.66 \times 3 = 0.54$$

$$Y= 0.54+0.66 \times x$$

$$\text{For } x=7 \\ = 5.16$$

Linear Regression in Matrix Form

$$\bullet \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{pmatrix}$$

- $Y = Xa + e$,
- $X = n \times 2$
- $Y = n \times 1$
- $A = 2 \times 1$
- $e = n \times 1$

Xi (Week)	Yi (Prod Sales)
1	1
2	3
3	4
4	8

Xi (Week)	Yi (Prod Sales)
1	1
2	3
3	4
4	8

$$X^t = [1 \ 2 \ 3 \ 4], \quad Y^t = [1 \ 3 \ 4 \ 8]$$

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}$$

$$Y = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 8 \end{pmatrix}$$

$$a = ((X^t X)^{-1} X^t) Y$$

After Calculating a,

a0 = Intercept = -1.5000

a1 = Slope = 2.2000

```
>> X'*X
```

```
ans =
```

```
4    10
```

```
10    30
```

```
>> inv(X'*X)
```

```
ans =
```

```
1.5000 -0.5000
```

```
-0.5000 0.2000
```

```
>> inv(X'*X)*X'
```

```
ans =
```

```
1.0000 0.5000 0 -  
0.5000
```

```
-0.3000 -0.1000 0.1000  
0.3000
```

```
>> (inv(X'*X)*X')*Y
```

```
ans =
```

```
-1.5000
```

```
2.2000
```

After calculating a,

a0 = Intercept = -1.5000

a1 = Slope = 2.2000

Y = -1.5 + 2.2X

Validation of Regression Model

xi	yi
I1	80
I2	90
I3	100
I4	110
I5	120

a) $MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$

• y_i = actual, $\hat{y}_i$ = Predicted

b) $MSE_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$

• Root Mean Square Error RMSE

c) $RMSE = \sqrt{MSE}$

d) Relative MSE

• $RelMSE = \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

0 – Model is perfect and its value between 0 to 1

1 not good

e) Coeff of CV

$$CV = \frac{RMSE}{\bar{y}}$$

Let I6 & I7 – Actual values are 80 & 75
Regression Predicts – 75 & 85

	yi	$\hat{y}_i$
I6	80	75
I7	75	85

$$\bar{y} = (80+90+100+110+120)/5 = 100$$

$$RelMSE = \frac{(80-75)^2 + (75-85)^2}{(80-100)^2 + (75-100)^2} = .1219$$

$$CV = .08$$

Validation of Regression Model

- a) $MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$
- y_i = actual, $\hat{y}_i$ = Predicted
- b) $MSE_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$
- Root Mean Square Error RMSE
- c) $RMSE = \sqrt{MSE}$
- d) Relative MSE
- $RelMSE = \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$
- 0 – Model is perfect and its value between 0 to 1
- 1 not good
- e) Coeff of CV
- $$CV = \frac{RMSE}{\bar{y}}$$

xi	yi
I1	80
I2	90
I3	100
I4	110
I5	120

Let I6 & I7 – Actual values are 80 & 75
Regression Predicts – 75 & 85

	yi	$\hat{y}_i$
I6	80	75
I7	75	85

$$MAE = \frac{1}{2} * (|80-75| + |75-85|) = 15/2 = 7.5$$

$$MSE = 62.5$$

$$RMSE = \sqrt{62.5} = 7.91$$

Validation of Regression Model

Coeff of determination

$$r^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$SS_{res} = \sum (\hat{y}_i - \bar{y})^2$$

$$SS_{tot} = \sum (y_i - \hat{y})^2$$

$$r^2$$

1 to inf

Standard Error Estimation:

$$S_e = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{n - 2}}$$

Multiple Linear Regression

- $Y = f(x_1, x_2)$

$$= a_0 + a_1 x_1 + a_2 x_2 + \dots + a_n x_n + \epsilon$$

Where $(x_1, \dots, x_n)$ are independent variables

Y is dependent

$(a_0, \dots, a_n)$ – Coef of regression coefficient

x1	x2	yi
1	4	1
2	5	6
3	8	8
4	2	12

X =

```

1  1  4
1  2  5
1  3  8
1  4  2

```

>> Y= [1 6 8 12]'

Y =

```

1
6
8
12

```

$$a = ((X^t X)^{-1} X^t) Y$$

>> a= (inv(X'*X)*X')*Y

a =

```

-1.6995
 3.4836
-0.0546

```

Bayesian Theorem

- Let the training dataset $\mathbf{D}$ consist of n points $\mathbf{x}_i$ in a d -dimensional space, and let y_i denote the class for each point,
- with $y_i \in \{c_1, c_2, \dots, c_k\}$
- The Bayes classifier directly uses the Bayes theorem to predict the class for a new test instance, $\mathbf{x}$.
- It estimates the posterior probability $P(c_i | \mathbf{x})$ for each class c_i , and chooses the class that has the largest probability.
- The predicted class for $\mathbf{x}$ is given as
- $\hat{y} = \mathit{argmax}_{c_i} \{ P(c_i | \mathbf{x}) \}$

Bayesian Theorem

- $\hat{y} = \mathop{\text{argmax}}_{c_i} \{ P(c_i | X) \}$
- The Bayes theorem allows us to invert the posterior probability in terms of the likelihood and prior probability, as follows:
- $$P(c_i | X) = \frac{P(X|c_i) \cdot P(c_i)}{P(X)}$$
- $P(X|c_i)$ is the *likelihood*, defined as probability of observing $\mathbf{x}$ assuming that the true class is c_i .
- $P(c_i)$ is the *prior probability* of class c_i , and $P(\mathbf{x})$ is the probability
- of observing $\mathbf{x}$ from any of the k classes, given as
- $$P(X) = \sum_{j=1}^k P(X|c_j) \cdot P(c_j)$$

Bayesian Theorem

$$\bullet \hat{y} = \mathop{arg}_{c_i}^{max} \{ P(c_i | X) \}$$

$$= \mathop{arg}_{c_i}^{max} \frac{P(X|c_i) \cdot P(c_i)}{P(X)}$$

$$= \mathop{arg}_{c_i}^{max} P(X|c_i) \cdot P(c_i)$$

- the predicted class essentially depends on the likelihood of that class
- taking its prior probability into account.
- For class c_i , the probability density at $\mathbf{x}$ is thus given as

$$f_i(\mathbf{x}) = f(\mathbf{x}|\mu_i, \Sigma_i) = \frac{1}{(\sqrt{2\pi})^d \sqrt{|\Sigma_i|}} \exp \left\{ -\frac{(\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i)}{2} \right\}$$

BAYESCLASSIFIER ($\mathbf{D} = \{(\mathbf{x}_j, y_j)\}_{j=1}^n$):

```
1 for  $i = 1, \dots, k$  do
2    $\mathbf{D}_i \leftarrow \{\mathbf{x}_j \mid y_j = c_i, j = 1, \dots, n\}$  // class-specific subsets
3    $n_i \leftarrow |\mathbf{D}_i|$  // cardinality
4    $\hat{P}(c_i) \leftarrow n_i/n$  // prior probability
5    $\hat{\boldsymbol{\mu}}_i \leftarrow \frac{1}{n_i} \sum_{\mathbf{x}_j \in \mathbf{D}_i} \mathbf{x}_j$  // mean
6    $\mathbf{Z}_i \leftarrow \mathbf{D}_i - \mathbf{1}_{n_i} \hat{\boldsymbol{\mu}}_i^T$  // centered data
7    $\hat{\boldsymbol{\Sigma}}_i \leftarrow \frac{1}{n_i} \mathbf{Z}_i^T \mathbf{Z}_i$  // covariance matrix
8 return  $\hat{P}(c_i), \hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i$  for all  $i = 1, \dots, k$ 
```

TESTING ($\mathbf{x}$ and $\hat{P}(c_i), \hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i$, for all $i \in [1, k]$):

```
9  $\hat{y} \leftarrow \underset{c_i}{\operatorname{argmax}} \{ f(\mathbf{x} | \hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i) \cdot P(c_i) \}$ 
10 return  $\hat{y}$ 
```

Example

- Consider the 2-dimensional Iris data, with attributes sepal length and sepal width. Class c_1 , which corresponds to iris-setosa has $n_1 = 50$ points, whereas the other class c_2 has $n_2 = 100$ points.

The prior probabilities for the two classes are

$$\hat{P}(c_1) = \frac{n_1}{n} = \frac{50}{150} = 0.33$$

$$\hat{P}(c_2) = \frac{n_2}{n} = \frac{100}{150} = 0.67$$

$$\hat{\mu}_1 = \begin{pmatrix} 5.01 \\ 3.42 \end{pmatrix}$$

$$\hat{\mu}_2 = \begin{pmatrix} 6.26 \\ 2.87 \end{pmatrix}$$

$$\hat{P}(c_1) = \frac{n_1}{n} = \frac{50}{150} = 0.33$$

$$\hat{P}(c_2) = \frac{n_2}{n} = \frac{100}{150} = 0.67$$

$$\hat{\mu}_1 = \begin{pmatrix} 5.01 \\ 3.42 \end{pmatrix}$$

$$\hat{\mu}_2 = \begin{pmatrix} 6.26 \\ 2.87 \end{pmatrix}$$

$$\hat{\Sigma}_1 = \begin{pmatrix} 0.122 & 0.098 \\ 0.098 & 0.142 \end{pmatrix}$$

$$\hat{\Sigma}_2 = \begin{pmatrix} 0.435 & 0.121 \\ 0.121 & 0.110 \end{pmatrix}$$

Let $\mathbf{x} = (6.75, 4.25)^T$ be a test point (shown as white square).

The posterior probabilities for c_1 and c_2 can be computed using

$$\hat{P}(c_1|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_1, \hat{\Sigma}_1) \hat{P}(c_1) = (4.914 \times 10^{-7}) \times 0.33 = 1.622 \times 10^{-7}$$

$$\hat{P}(c_2|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_2, \hat{\Sigma}_2) \hat{P}(c_2) = (2.589 \times 10^{-5}) \times 0.67 = 1.735 \times 10^{-5}$$

Because $\hat{P}(c_2|\mathbf{x}) > \hat{P}(c_1|\mathbf{x})$ the class for $\mathbf{x}$ is predicted as $\hat{y} = c_2$.

Challenges

- for numeric attributes we have to estimate $O(d^2)$ covariances, and as the dimensionality increases, this requires us to estimate too many parameters.

NAIVE BAYES CLASSIFIER

- The naive Bayes approach makes the simple assumption that all the attributes are independent.
- Training the naive Bayes classifier is very fast, with $O(nd)$ computational complexity.

NAIVE BAYES CLASSIFIER

$$\hat{\Sigma}_1 = \begin{pmatrix} 0.122 & 0 \\ 0 & 0.142 \end{pmatrix}$$

$$\hat{\Sigma}_2 = \begin{pmatrix} 0.435 & 0 \\ 0 & 0.110 \end{pmatrix}$$

For the test point $\mathbf{x} = (6.75, 4.25)^T$, the posterior probabilities for c_1 and c_2 are as follows:

$$\hat{P}(c_1|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_1, \hat{\Sigma}_1) \hat{P}(c_1) = (3.99 \times 10^{-7}) \times 0.33 = 1.32 \times 10^{-7}$$

$$\hat{P}(c_2|\mathbf{x}) \propto \hat{f}(\mathbf{x}|\hat{\mu}_2, \hat{\Sigma}_2) \hat{P}(c_2) = (9.597 \times 10^{-5}) \times 0.67 = 6.43 \times 10^{-5}$$

Because $\hat{P}(c_2|\mathbf{x}) > \hat{P}(c_1|\mathbf{x})$ the class for $\mathbf{x}$ is predicted as $\hat{y} = c_2$.

NAIVE BAYES CLASSIFIER

$$\mathbf{\Sigma}_i = \begin{pmatrix} \sigma_{i1}^2 & 0 & \dots & 0 \\ 0 & \sigma_{i2}^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \dots & \sigma_{id}^2 \end{pmatrix}$$

This yields

$$|\mathbf{\Sigma}_i| = \det(\mathbf{\Sigma}_i) = \sigma_{i1}^2 \sigma_{i2}^2 \cdots \sigma_{id}^2 = \prod_{j=1}^d \sigma_{ij}^2$$

Also, we have

$$\mathbf{\Sigma}_i^{-1} = \begin{pmatrix} \frac{1}{\sigma_{i1}^2} & 0 & \dots & 0 \\ 0 & \frac{1}{\sigma_{i2}^2} & \dots & 0 \\ \vdots & \vdots & \ddots & \\ 0 & 0 & \dots & \frac{1}{\sigma_{id}^2} \end{pmatrix}$$

Training dataset

Class:

C1:buys_computer='yes'

C2:buys_computer='no'

Data sample

X =(age<=30,

Income=medium,

Student=yes

Credit_rating=Fair)

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
30...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

- Let C_1 correspond to the class buys computer = yes and C_2 correspond to buys computer = no. The unknown sample we wish to classify is
- $X = (\text{age} = "<30", \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit rating} = \text{fair})$
- We need to maximize $P(X|C_i)P(C_i)$, for $i = 1, 2$. $P(C_i)$, the prior probability of each class, can be computed based on the training samples:
- $P(\text{buys computer} = \text{yes}) = 9/14 = 0.643$
- $P(\text{buys computer} = \text{NO}) = 5/14 = 0.357$

Naïve Bayesian Classifier: Example

- Compute $P(X/C_i)$ for each class

$$P(\text{age} = "<30" \mid \text{buys_computer} = \text{"yes"}) = 2/9 = 0.222$$

$$P(\text{age} = "<30" \mid \text{buys_computer} = \text{"no"}) = 3/5 = 0.6$$

$$P(\text{income} = \text{"medium"} \mid \text{buys_computer} = \text{"yes"}) = 4/9 = 0.444$$

$$P(\text{income} = \text{"medium"} \mid \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

$$P(\text{student} = \text{"yes"} \mid \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{student} = \text{"yes"} \mid \text{buys_computer} = \text{"no"}) = 1/5 = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} \mid \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{credit_rating} = \text{"fair"} \mid \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

K NEAREST NEIGHBORS CLASSIFIER

- **non-parametric approach,**
- which does not make any assumptions about the underlying joint probability density function.
- it directly uses the data sample to estimate the density,
- Let **$\mathbf{D}$** be a training dataset comprising n points $\mathbf{x}_i \in \mathbb{R}^d$,
- and let **$\mathbf{D}_i$** denote the subset of points in **$\mathbf{D}$** that are labeled with class c_i , with $n_i = |\mathbf{D}_i|$.
- Given a test point $\mathbf{x} \in \mathbb{R}^d$, and K , the number of neighbors to consider,
- let r denote the distance from $\mathbf{x}$ to its K th nearest neighbor in **$\mathbf{D}$** .

K NEAREST NEIGHBORS CLASSIFIER

- Consider the d-dimensional hyperball of radius r around the test point $\mathbf{x}$, defined as
- $\mathbf{B}_d(\mathbf{x}, r) = \{\mathbf{x}_i \in \mathbf{D} \mid \delta(\mathbf{x}, \mathbf{x}_i) \leq r\}$
- $\delta(\mathbf{x}, \mathbf{x}_i)$ is the distance between $\mathbf{x}$ and $\mathbf{x}_i$, which is usually assumed to be the Euclidean distance,
- $\delta(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2$
- Let K_i denote the number of points among the K nearest neighbors of $\mathbf{x}$ that are labeled with class c_i ,

$$K_i = \#\{ \mathbf{x}_j \in \mathbf{B}_d(\mathbf{x}, r) \mid \mathbf{y}_j = c_i \}$$

K NEAREST NEIGHBORS CLASSIFIER

- The class conditional probability density at $\mathbf{x}$ can be estimated as the fraction of points from class c_i that lie within the hyperball divided by its volume.

$$\hat{f}(\mathbf{x}|c_i) = \frac{K_i/n_i}{V} = \frac{K_i}{n_i V}$$

- where $V = \text{vol}(B_d(\mathbf{x}, r))$ is the volume of the d -dimensional hyperball.
- posterior probability $P(c_i | \mathbf{x})$ can be estimated as

$$P(c_i|\mathbf{x}) = \frac{\hat{f}(\mathbf{x}|c_i) \hat{P}(c_i)}{\sum_{j=1}^k \hat{f}(\mathbf{x}|c_j) \hat{P}(c_j)}$$

K NEAREST NEIGHBORS CLASSIFIER

- However, because $\hat{P}(c_i) = n_i / n$, we have

$$\hat{f}(\mathbf{x}|c_i)\hat{P}(c_i) = \frac{K_i}{n_i V} \cdot \frac{n_i}{n} = \frac{K_i}{nV}$$

Thus the posterior probability is given as

$$P(c_i|\mathbf{x}) = \frac{\frac{K_i}{nV}}{\sum_{j=1}^K \frac{K_j}{nV}} = \frac{K_i}{K}$$

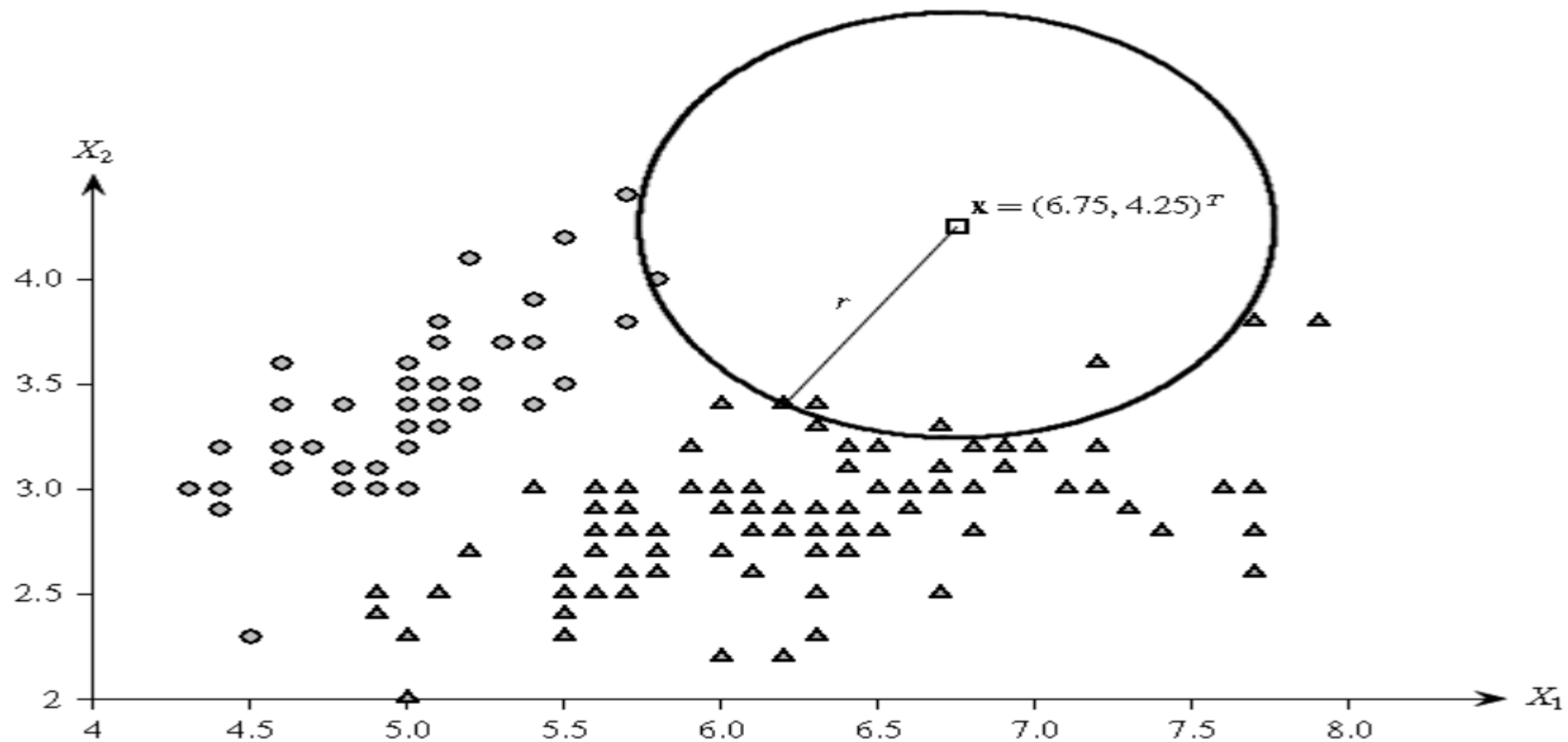
Finally, the predicted class for $\mathbf{x}$ is

$$\hat{y} = \arg\max_{c_i} \{P(c_i|\mathbf{x})\} = \arg\max_{c_i} \left\{ \frac{K_i}{K} \right\} = \arg\max_{c_i} \{K_i\}$$

Example

- Consider the 2D Iris dataset . The two classes are: c_1 (circles) with $n_1 = 50$ points and c_2 (triangles) with $n_2 = 100$ points.
- Let us classify the test point $\mathbf{x} = (6.75, 4.25)^T$ using its $K = 5$ nearest neighbors, is given as $r = \sqrt{1.025} = 1.012$
- It encompasses $K_1 = 1$ point from class c_1
- and $K_2 = 4$ points from class c_2 .
- **Therefore, the predicted class for $\mathbf{x}$ is $\hat{y} = c_2$.**

Example



Capacity, Overfitting and Underfitting

- The ability to perform well on previously unobserved inputs is called **generalization**.
- when training a machine learning model, we have access to a **training set**, we can compute some error measure on training set called the **training error**, and we **reduce this training error**.
- This is an **optimization problem**.
- What separates machine learning from optimization is that we want the **generalization error, also called the test error, to be low** as well.
- **generalization error** is the expected value of error on a **new input**

Capacity, Overfitting and Underfitting

- we trained the model by minimizing the training error,
- $\frac{1}{m^{(train)}} \|X^{(train)}\mathbf{w} - \mathbf{y}^{(train)}\|_2^2$

but we actually care about the test error,

- $\frac{1}{m^{(test)}} \|X^{(test)}\mathbf{w} - \mathbf{y}^{(test)}\|_2^2$

How can we affect performance on the test set when we get to observe only the training set?

Capacity, Overfitting and Underfitting

- The train and test data are generated by a probability distribution over datasets called the data generating process.
- We typically make a set of assumptions known collectively as the **i.i.d. assumptions**.
- These assumptions are that the examples in each dataset are independent from each other,
- and that the **train set and test set** are **identically distributed**, drawn from the same probability distribution as each other.

Capacity, Overfitting and Underfitting

- The factors determining how well a machine learning algorithm will perform are its ability to:
 1. **Make the training error small.**
 2. **Make the gap between training and test error small.**
- These two factors correspond to the two central challenges in machine learning:
- **underfitting and overfitting .**
- **Underfitting occurs** when the model is not able to obtain a sufficiently **low error value on the training set.**
- **Overfitting occurs** when the gap between the **training error and test error** is too large.

Capacity, Overfitting and Underfitting

- A model is said to be a good machine learning to make predictions about future data.
- overfitting and underfitting, which are majorly responsible for the **poor performances**.

underfitting :

- a model is too simple to capture data complexities.
- inability of the model to learn the training data effectively result in poor performance **both on the training and testing data**.
- Models are inaccurate
- size of the training dataset used is not enough.

Overfitting

- model does not make accurate predictions on testing data.
- When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set.

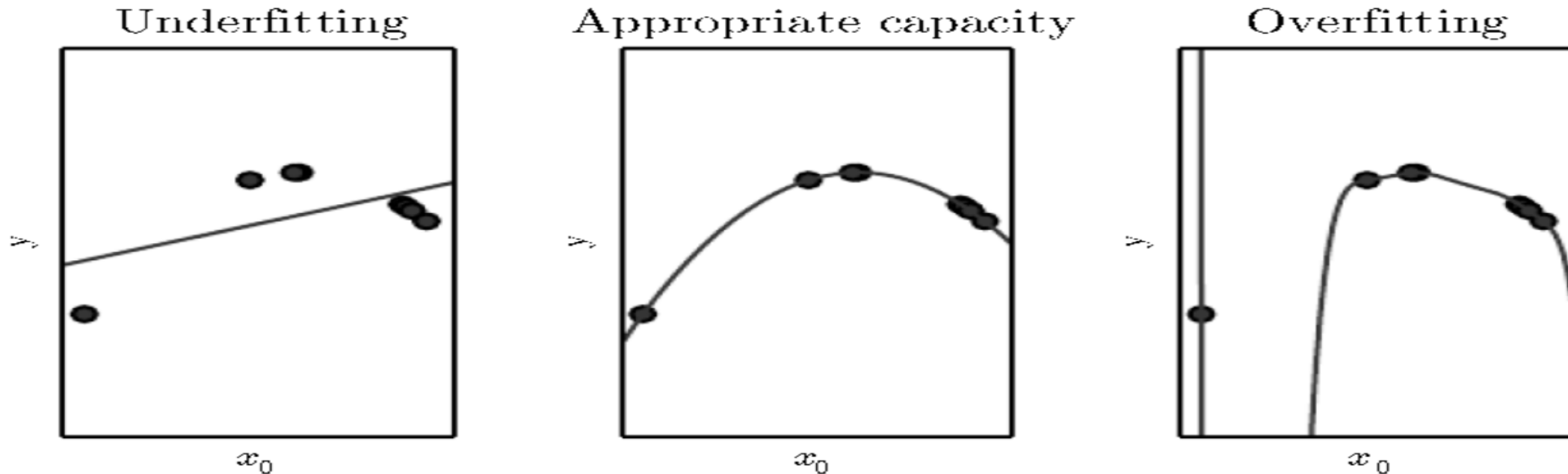
Capacity, Overfitting and Underfitting

- We can control whether a model is more likely to overfit or underfit by **altering its capacity**.
- Models with **low capacity** may struggle to fit the training set.
- Models with **high capacity** can overfit by memorizing properties of the training set that do not serve them well on the test set.
- We can generalize linear regression to include polynomials, rather than just linear functions, in its hypothesis space. Doing so increases the model's capacity.
- $\hat{y} = b + wx$
- $\hat{y} = b + w_1x + w_2x^2$
- obtain a polynomial of degree 9:
- $\hat{y} = b + \sum_{i=1}^9 w_i x^i$

Capacity, Overfitting and Underfitting

- Models with insufficient capacity are unable to solve complex tasks.
- Models with high capacity can solve complex tasks,
 - but when their capacity is higher than they may overfit.
- We compare a linear, quadratic and degree-9 predictor attempting to fit a problem where the true underlying function is quadratic.
- The linear function is **unable to capture the curvature** in the true underlying problem, so it underfits.
- The **degree-9 predictor** is capable of representing the correct function, but it is also capable of representing infinitely many other functions that pass exactly through the training points,

Capacity, Overfitting and Underfitting



(Left) A linear function fit to the data suffers from underfitting—it cannot capture the curvature that is present in the data.

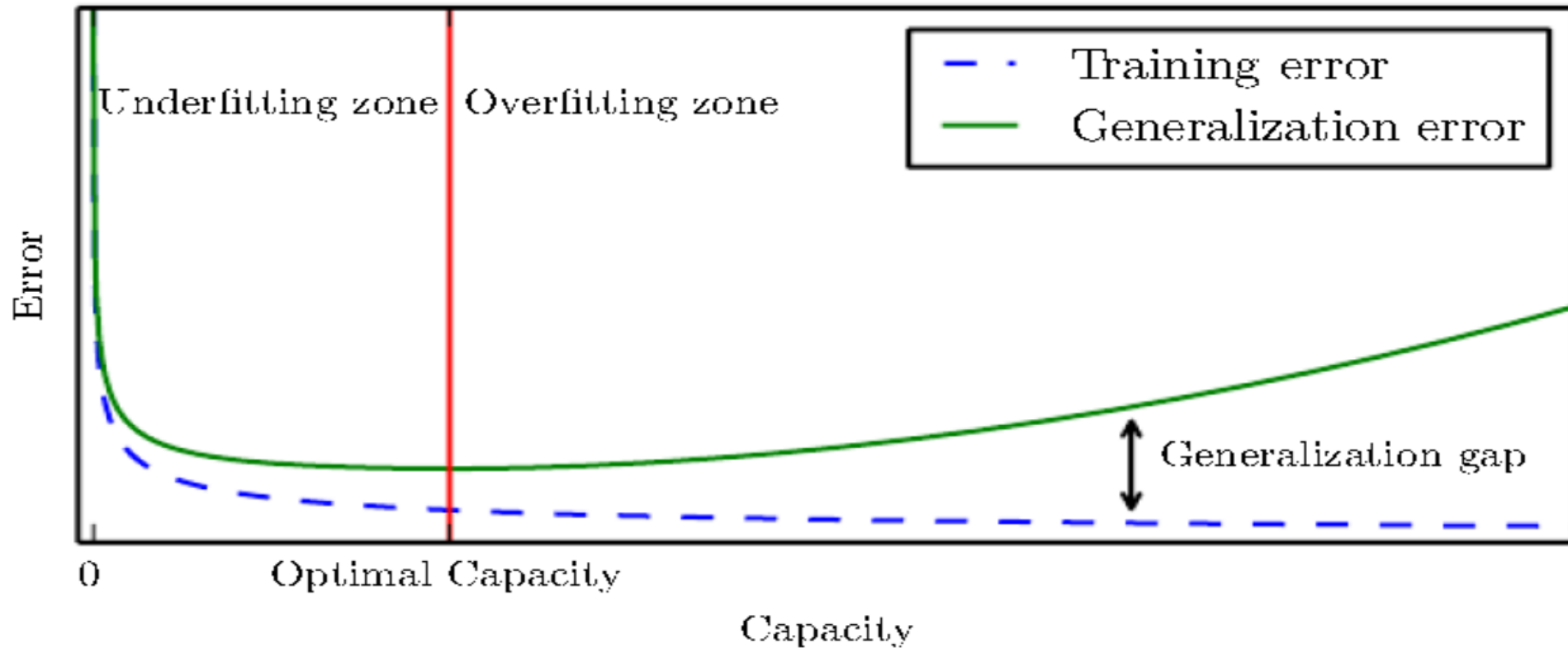
(Center) A quadratic function fit to the data generalizes well to unseen points. It does not suffer from a significant amount of overfitting or underfitting.

(Right) A polynomial of degree 9 fit to the data suffers from overfitting

Capacity, Overfitting and Underfitting

- one way of changing a model's capacity:
- by changing the number of input features it has,
- and simultaneously adding new parameters associated with those features.
- In many cases, finding the best function within this family is a very difficult optimization problem.
- learning algorithm does not actually find the best function, but merely one that significantly reduces the training error.

Capacity, Overfitting and Underfitting



Capacity, Overfitting and Underfitting

- Training and test error behave differently.
- At the left end of the graph, **training error and generalization error are both high. This is the underfitting regime.**
- **As we increase capacity, training error decreases, but the gap between training and generalization error increases.**

Capacity, Overfitting and Underfitting

- **Non-parametric models have no such limitation.**
- One example of such an algorithm is **nearest neighbor regression**.
- Unlike linear regression, which has a fixed-length vector of weights,
- the nearest neighbor regression model simply stores the **X and y from the training set**.
- When asked to classify a test point x , the model looks up the nearest entry in the training set and returns the associated regression target.
- $\hat{y} = y_i$ **where $i = \arg \min \|X_i - x\|_2^2$**

Capacity, Overfitting and Underfitting

- The ideal model is an **oracle** that simply knows the true **probability distribution that generates the data**.
- Even such a model **will still incur some error** on many problems, because there may still be **some noise in the distribution**.

The error incurred by an oracle making predictions from the true distribution $p(x, y)$ is called the **Bayes error**.

Capacity, Overfitting and Underfitting

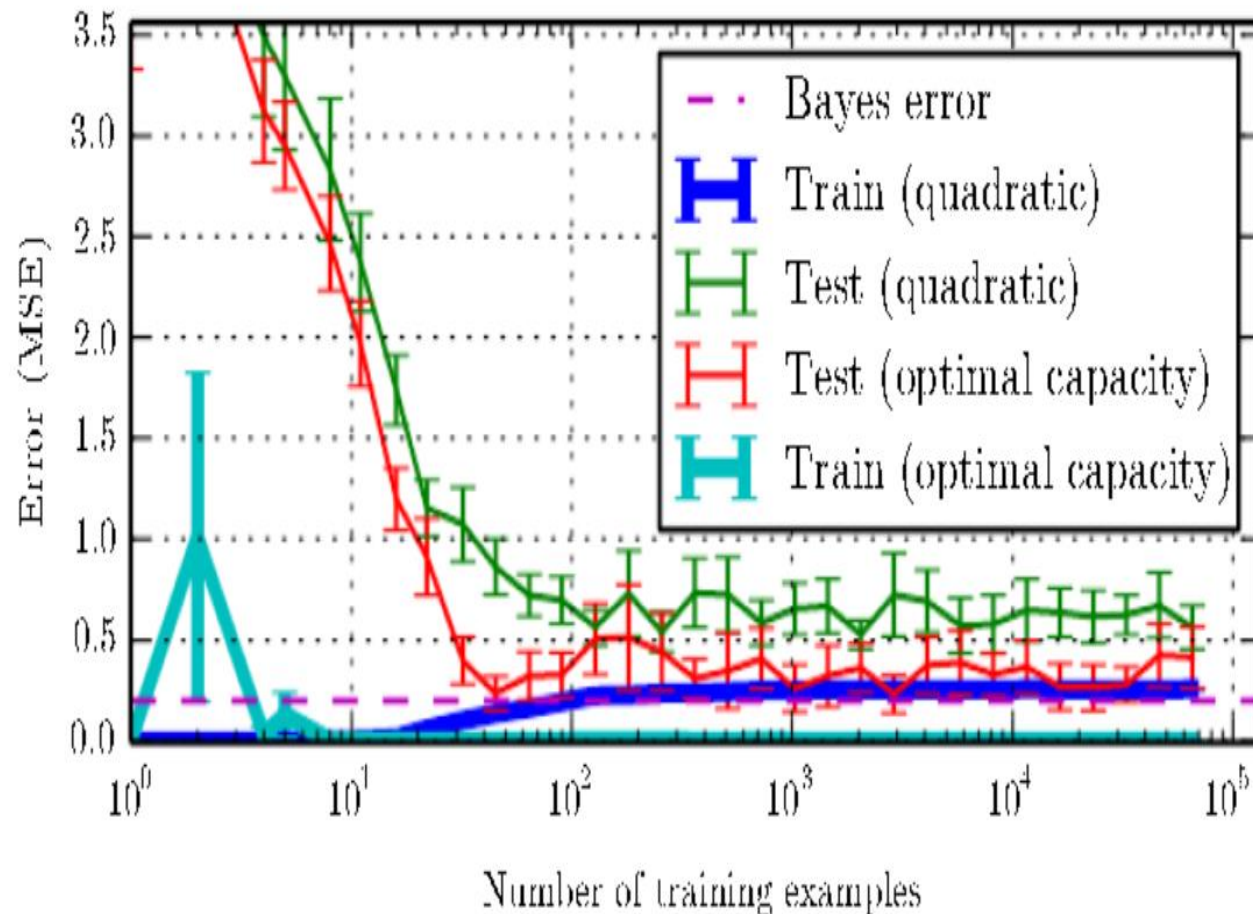
- Training and generalization error vary as the size of the training set varies.
- Expected generalization error can never increase as the number of training examples increases.
- For non-parametric models, more data yields better generalization until the best possible error is achieved.

Any fixed parametric model with less than optimal capacity will asymptote to an error value that exceeds the Bayes error.

Capacity, Overfitting and Underfitting

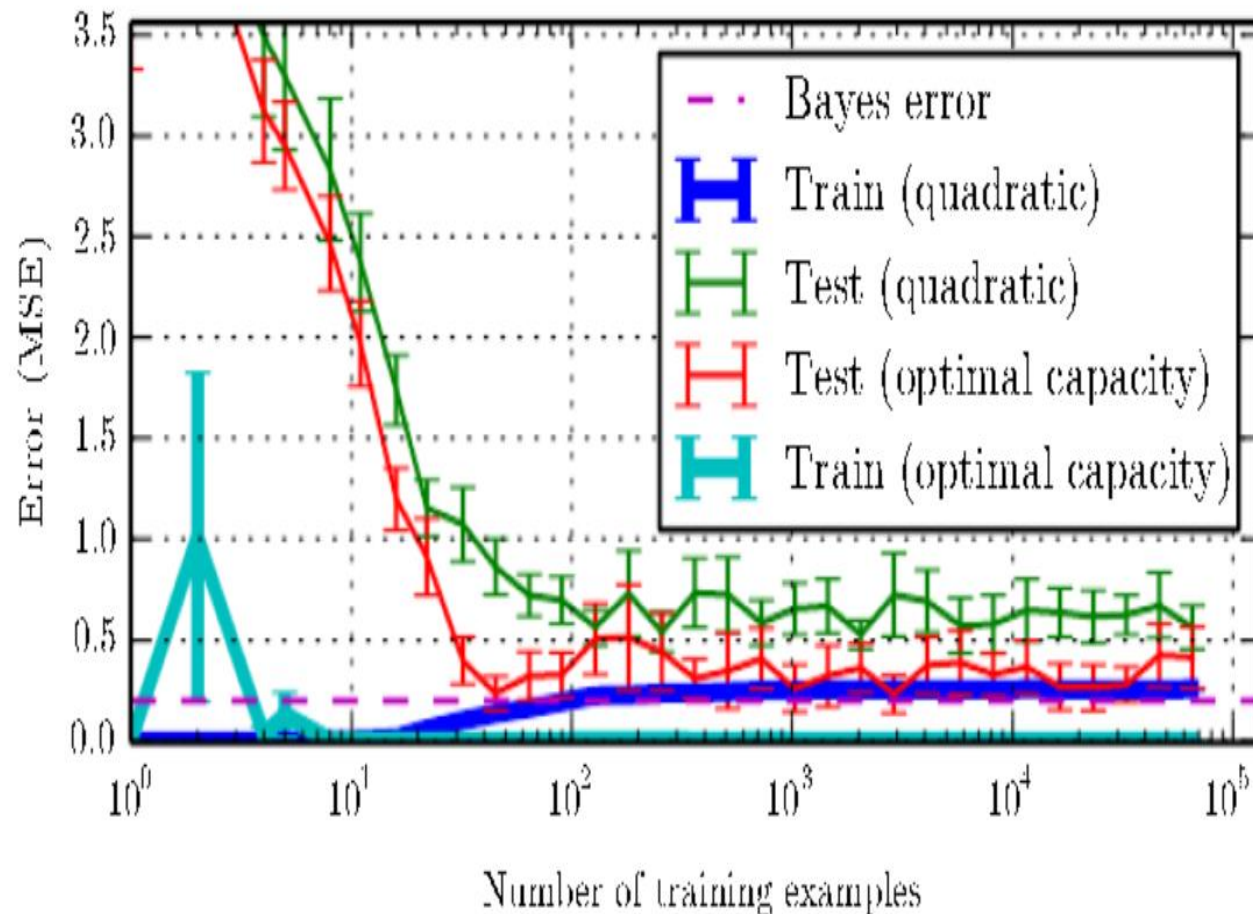
- it is possible for the model to have **optimal capacity** and yet still have a **large gap between training and generalization error**.
- we may be able to reduce this gap by gathering more training examples.
- **The No Free Lunch Theorem**
- every classification algorithm has the **same error rate** when classifying previously unobserved points.
- **no machine learning algorithm is universally any better than any other.**

Capacity, Overfitting and Underfitting



For the quadratic model,
training error increases as size of the training set increases.
test error decreases, because fewer incorrect hypotheses are consistent with the training data.
The quadratic model **does not have enough capacity** to solve the task, so its test error asymptotes to a **high value**.

Capacity, Overfitting and Underfitting



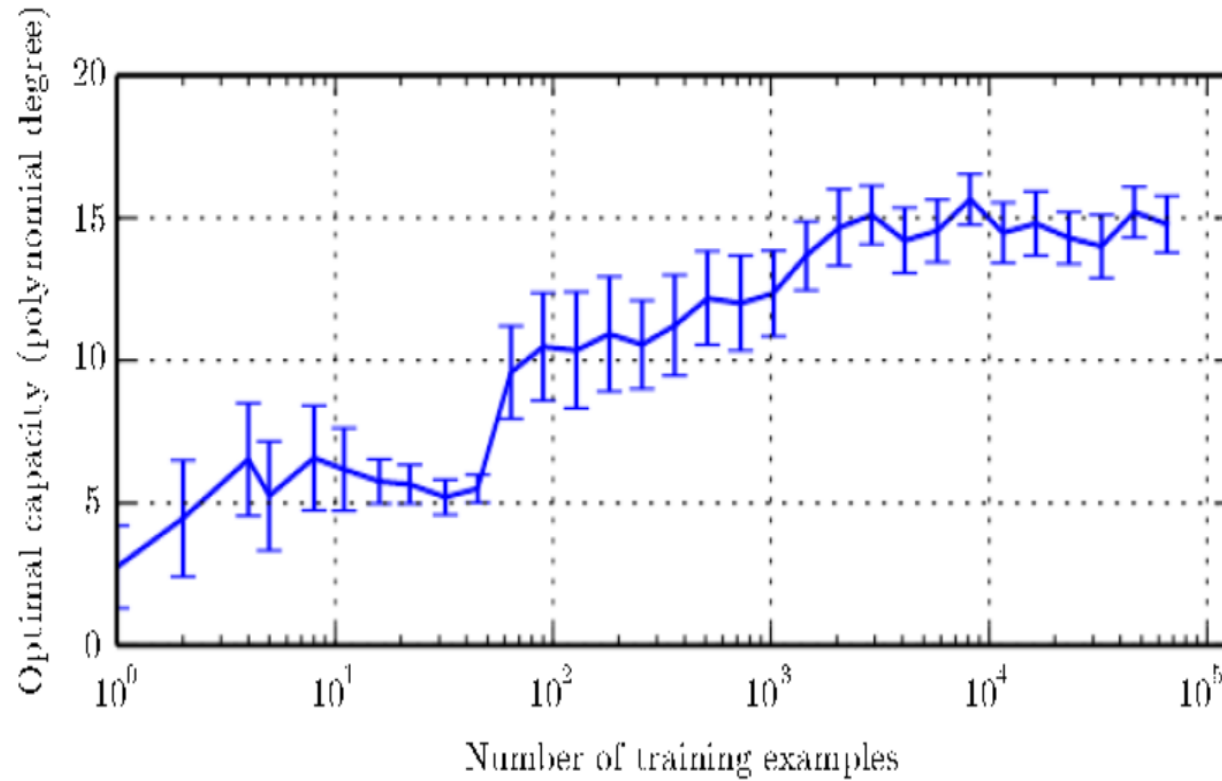
test error at optimal capacity

asymptotes to Bayes error.

training error can fall below **Bayes error**,

As **training size** increases to **infinity**,
training error of any fixed-capacity
model **quadratic model**) must rise to
at least Bayes error

Capacity, Overfitting and Underfitting



As the training set size increases, the optimal capacity (degree of the optimal polynomial regressor) increases.

goal of machine learning research is not to seek a universal learning algo or absolute best learning algo. Instead, our goal is to understand what kinds of distributions are relevant to the “real world” .

Capacity, Overfitting and Underfitting

- **Regularization**

- no free lunch theorem implies that we must design our machine learning algorithms to perform well on a specific task.
- **We do so by building a set of preferences into the learning algorithm.**
- linear functions can be very useful for problems where the relationship between inputs and outputs truly is close to linear.
- They are less useful for problems that behave in a very nonlinear fashion.

For example,

linear regression not perform very well if we tried to use it to predict **$\sin(x)$** from **x** .

We can **control the performance of** our algorithms by choosing what kind of functions we allow them to draw solutions from,
as well as by controlling the amount of these functions.

Capacity, Overfitting and Underfitting

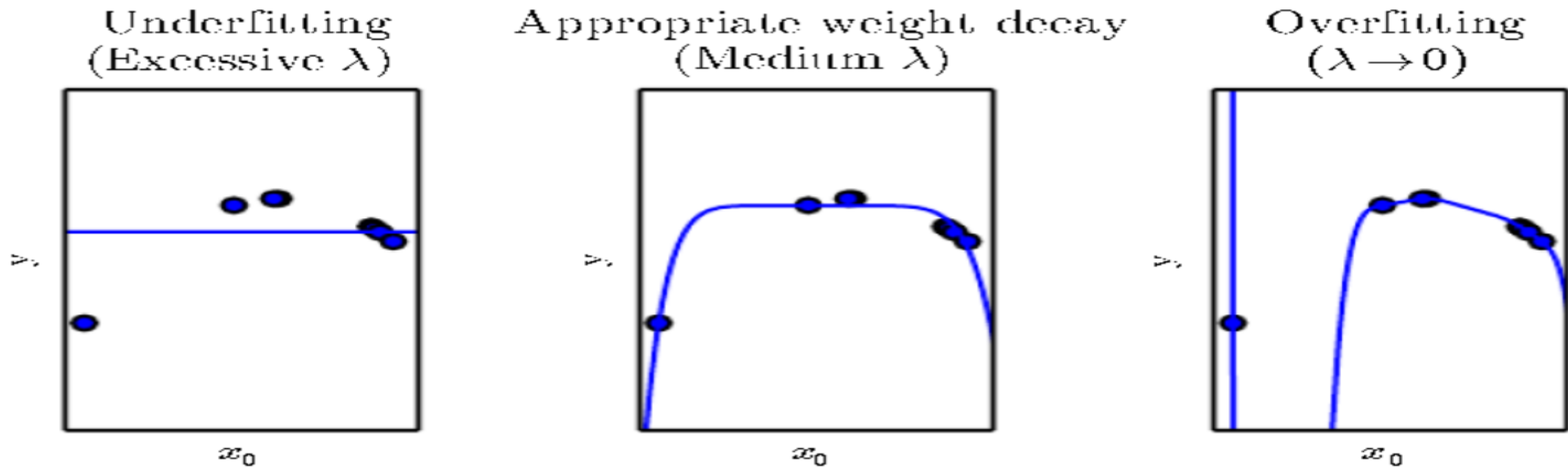
- **Regularization**

- we can modify the training criterion for linear regression to **include weight decay**.

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

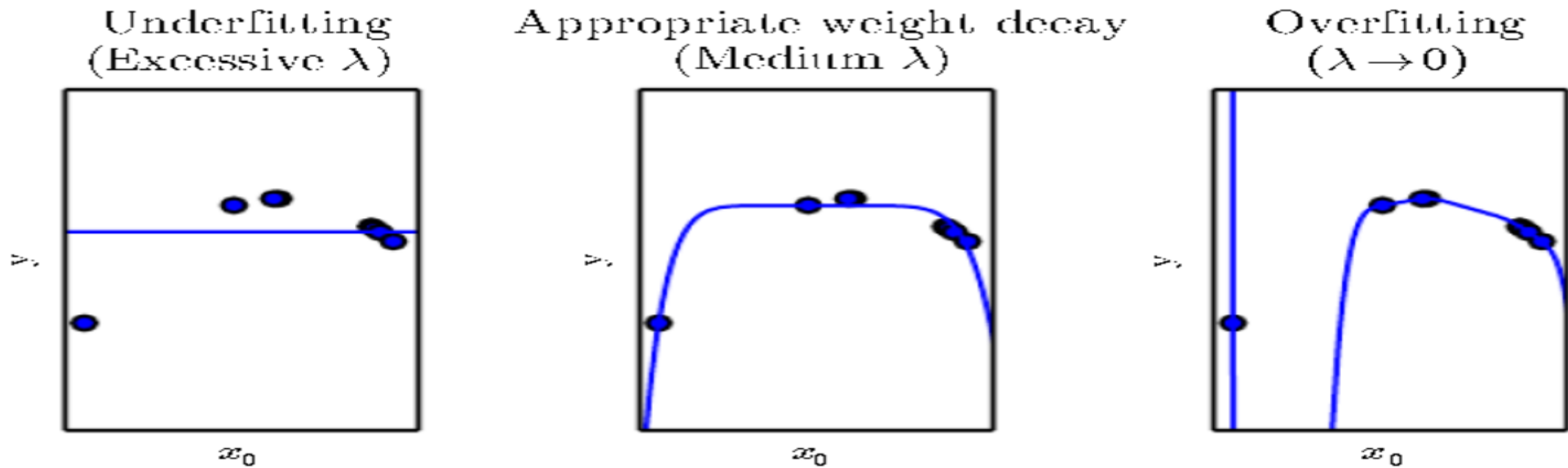
- where λ is a value that controls the strength of our preference for smaller weights.
- When $\lambda = 0$, we impose no preference,
- and **larger λ forces the weights to become smaller**.
- we can control a model's tendency to overfit or underfit via weight decay,
- we can train **high-degree polynomial** regression model **with different values of λ** .

Capacity, Overfitting and Underfitting



- **we use only models with degree 9.**
- (Left) With very large λ , we can force the model to learn a function with no slope at all.
- This underfits because it can only represent a constant function

Capacity, Overfitting and Underfitting



- *(Center)* With a medium value of λ , the learning algorithm recovers a curve with the right general shape.
- *(Right)* With weight decay approaching zero, the degree-9 polynomial overfits significantly

Capacity, Overfitting and Underfitting

- we can **regularize** a model that learns a function $f(x; \vartheta)$ by adding a penalty called a regularizer to the cost function.
- **In the case of weight decay, the regularizer is $\Omega(\mathbf{w}) = \mathbf{w}^T \mathbf{w}$.**
- *Regularization is any modification we make to a learning algorithm that is intended to reduce its **generalization error but not its training error**.*
- The no free lunch theorem has made it clear that there is no best machine learning algorithm, and no best form of regularization.

Instead we must choose a form of regularization that is well-suited to the particular task we want to solve

Hyperparameters and Validation Sets

- Most machine learning algorithms have **several settings**
- that we can use to control the behavior of the learning algorithm.
- These settings are called **hyperparameters**.
- **degree of the polynomial**, which acts as a **capacity hyperparameter**.
- **λ value** used to **control strength** of **weight decay** is example of **hyperparameter**
- it is not appropriate to learn that hyperparameter on the training set

Hyperparameters and Validation Sets

- Specifically, we split the **training data into two disjoint subsets**.
- One of **these subsets** is used to **learn the parameters**.
- The other subset is our **validation set**, used to estimate **the generalization error** during or after training, allowing for the **hyperparameters to be updated accordingly**.
- Typically, **one uses about 80% of the training data** for training
- **and 20% for validation**.
- Since the **validation set** is used to **“train” the hyperparameters**,
- the validation set error will underestimate the generalization error, by a smaller amount than the training error.
- After all hyperparameter optimization is complete, the generalization error may be estimated using the test set.

Hyperparameters and Validation Sets

- **A) Cross-Validation**

- Dividing the dataset into a **fixed training set** and a **fixed test set** can be problematic if it results in the **test set being small**.
- A small test set implies statistical uncertainty around the estimated average test error, making it difficult to claim that algorithm *A* works better than algorithm *B* on the given task.
- When the dataset has **hundreds of thousands of examples or more**, this is not a serious issue.
- **We get problems when the dataset is too small**, estimation of the mean **test error**, at the price of increased computational cost.

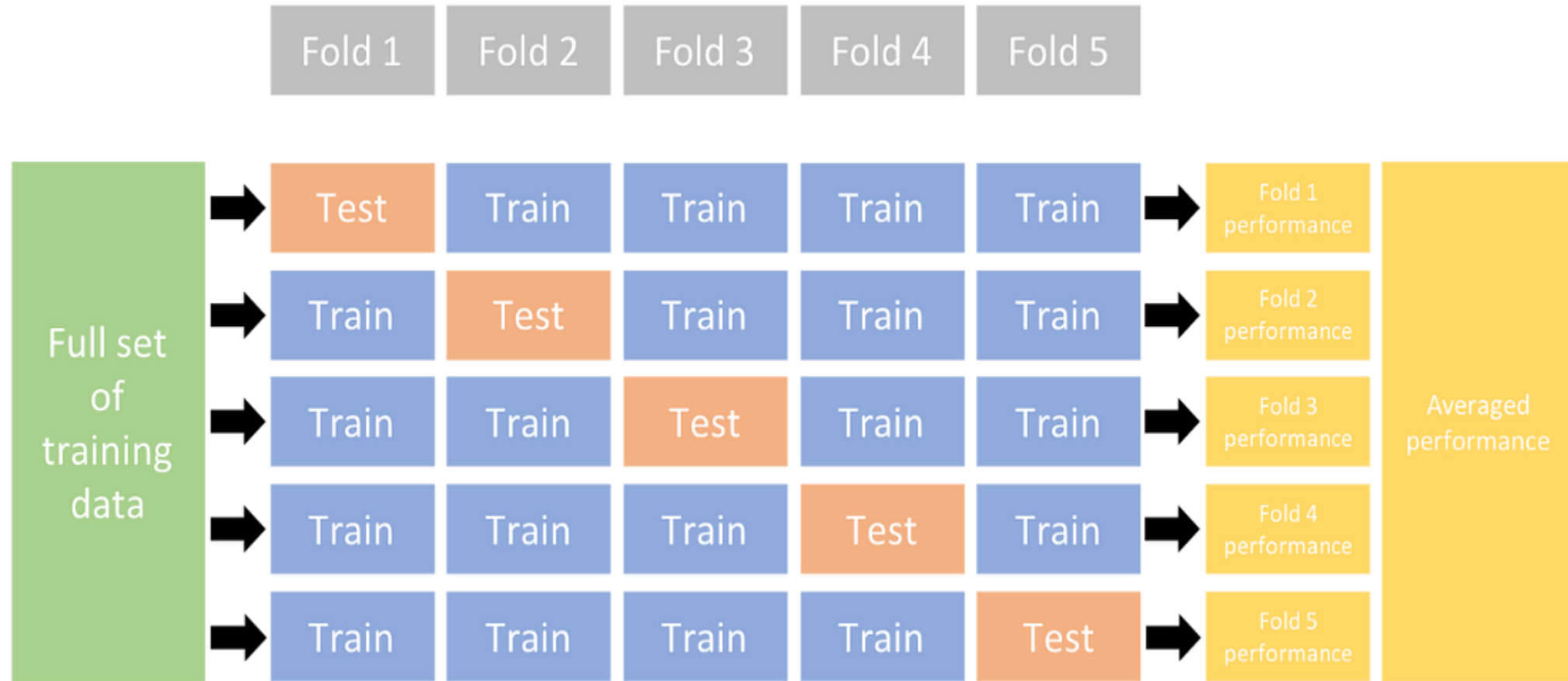
Hyperparameters and Validation Sets

A) Cross-Validation

- These procedures are based on the idea of repeating the training and testing computation on different randomly chosen subsets or splits of the original dataset.
- The most common of these is the ***k*-fold cross-validation** procedure

Hyperparameters and Validation Sets

A) Cross-Validation



Hyperparameters and Validation Sets

A) Cross-Validation

- **k -fold cross-validation** procedure.
- in which a partition of the dataset is formed by splitting it into **k non-overlapping subsets**.
- The test error may then be estimated by taking the average test error across k trials.
- On trial i , the i -th subset of the data is used as the test set and the rest of the data is used as the training set.

Estimators, Bias and Variance

- **parameter estimation, bias and variance** are useful to **generalization, underfitting and overfitting.**
- **Point Estimation**
- attempt to provide **the single “best” prediction.**
- can be **single parameter or a vector of parameters**
- such as the **weights in linear regression** but it can also be a **whole function.**
- denote a point estimate of a parameter θ by $\hat{\theta}$

Estimators, Bias and Variance

- **Point Estimation**
- **Let** $\{\mathbf{x}^{(1)}, \dots \dots \mathbf{x}^{(m)}\}$ be a set of m independent and identically distributed (i.i.d.) data points.
- A point estimator or statistic function of the data:
- $\widehat{\boldsymbol{\theta}}_m = \mathbf{g}(\mathbf{x}^{(1)}, \dots \dots \mathbf{x}^{(m)})$
- a good estimator is a function whose output is **close to the true** underlying $\boldsymbol{\theta}$ that generated the training data

Estimators, Bias and Variance

- **Point Estimation**
- we assume that the true parameter **value θ is fixed** but unknown, while the **point estimate $\hat{\theta}$ is a function of the data**.
- Since the data is drawn from a random process, any function of the data is random. Therefore $\hat{\theta}$ is a random variable.
- Point estimation can also refer to the estimation of the **relationship between input and target variables**

Estimators, Bias and Variance

- **Function Estimation**

- we are trying to predict a **variable y given an input vector x** .
- $y = f(x) + \epsilon$, where ϵ stands for part of y that is not predictable from x .
- we are interested in approximating f with a model or estimate $\hat{f}$.
- The linear regression example and the polynomial regression example are both examples that may be interpreted either as **estimating a parameter w**
- **or estimating a function $\hat{f}$ mapping from x to y .**

Estimators, Bias and Variance

Bias

- $\text{bias}(\hat{\theta}_m) = E(\hat{\theta}_m) - \theta$
- θ is true underlying value of θ used to define data generating distribution.
- $\hat{\theta}_m$ is said to **be unbiased** if $\text{bias}(\hat{\theta}_m) = 0$,
- which implies $E(\hat{\theta}_m) = \theta$

Bernoulli Distribution

- Consider a set of samples $\{x^{(1)}, \dots, x^{(m)}\}$
- that are independently and identically distributed according to a Bernoulli distribution with **mean θ** :
- $P(x^{(i)}; \theta) = \theta^{x^{(i)}} (1 - \theta)^{(1-x^{(i)})}$
- $\widehat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$
- $bias(\widehat{\theta}_m) = E(\widehat{\theta}_m) - \theta$
- $= E\left[\frac{1}{m} \sum_{i=1}^m x^{(i)}\right] - \theta$
- $\frac{1}{m} \sum_{i=1}^m E[x^{(i)}] - \theta$

Bernoulli Distribution

- $= E \left[\frac{1}{m} \sum_{i=1}^m x^{(i)} \right] - \theta$
- $\frac{1}{m} \sum_{i=1}^m E[x^{(i)}] - \theta$
- $\frac{1}{m} \sum_{i=1}^m \sum_{x^{(i)}=0}^1 (x^{(i)} \theta^{x^{(i)}} (1 - \theta)^{(1-x^{(i)})} - \theta)$
- $\frac{1}{m} \sum_{i=1}^m (\theta) - \theta$
- $= \theta - \theta = 0$
- Since $\text{bias}(\widehat{\theta}_m) = 0$, we say that our estimator $\widehat{\theta}$ is unbiased

Gaussian Distribution Estimator of the Mean

- Consider a set of samples $\{ \mathbf{x}^{(1)}, \dots \dots \mathbf{x}^{(m)} \}$
- that are independently and identically distributed according to a Gaussian distribution
- $\mathbf{P}(\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{x}^{(i)}; \mu, \sigma^2)$;
- where $i \in \{1, \dots, m\}$.
- Gaussian probability density function is given by
- $p(\mathbf{x}^{(i)}; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(\mathbf{x}^{(i)} - \mu)^2}{\sigma^2}\right)$
- $\widehat{\mu}_m = \frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)}$

Gaussian Distribution Estimator of the Mean

- $\text{bias}(\hat{\mu}_m) = E(\hat{\mu}_m) - \mu$
- $= E\left[\frac{1}{m} \sum_{i=1}^m x^{(i)}\right] - \mu$
- $\frac{1}{m} \sum_{i=1}^m E[x^{(i)}] - \mu$
- $\frac{1}{m} \sum_{i=1}^m (\mu) - \mu$
- $= \mu - \mu = 0$
- Since $\text{bias}(\hat{\mu}_m) = 0$, we say that our estimator $\hat{\mu}$ is **unbiased**.
- mean is an unbiased estimator of Gaussian mean parameter.

Estimators of Variance of a Gaussian Distrib

- The first estimator of σ^2 we consider is known as the **sample variance**
- $\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$
- $\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$
- ***bias*** $(\hat{\sigma}_m^2) = E(\hat{\sigma}_m^2) - \sigma^2$
- $E(\hat{\sigma}_m^2) = E\left[\frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2\right]$
- $= \frac{m-1}{m} \sigma^2$
- we conclude that the bias of $\hat{\sigma}_m^2$ is $-\sigma^2/m$, Therefore,
- the sample variance is a **biased estimator**.

The unbiased sample variance estimator

- $\hat{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$
- $E(\hat{\sigma}_m^2) = \sigma^2$
- $E(\hat{\sigma}_m^2) = E[\frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2]$
- $= \frac{m}{m-1} E(\hat{\sigma}_m^2)$

Since $E(\hat{\sigma}_m^2) = E[\frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2] = \frac{m-1}{m} \sigma^2$

- $= \frac{m}{m-1} (\frac{m-1}{m} \sigma^2)$
- $= \sigma^2$
- We have two estimators: one is biased and the other is not.
- While unbiased estimators are clearly desirable, they are not always the “best” estimators

Variance and Standard Error

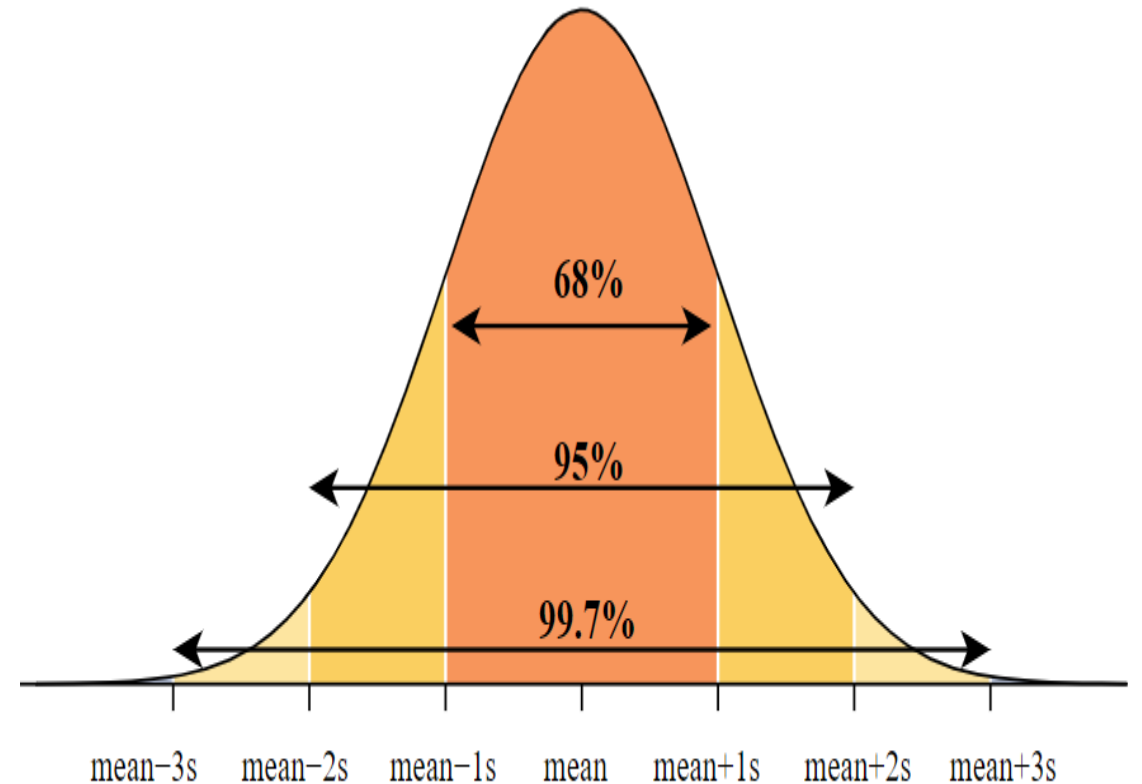
- how much we expect it to vary as a function of the data sample.
- The variance of an estimator is simply the variance. $\text{Var}(\hat{\theta})$
- where the random variable is the training set.
- square root of variance is called standard error denoted $\text{SE}(\hat{\theta})$.
- estimator to exhibit relatively low variance.
- $\text{SE}(\hat{\mu}_m) = \sqrt{\text{Var}\left[\frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)}\right]} = \frac{\sigma}{\sqrt{m}}$
- where σ^2 is the true variance of the samples $\mathbf{x}^{(i)}$.

Variance and Standard Error

- neither the **square root of the sample variance** nor the **square root of the unbiased estimator** of the variance provide an **unbiased estimate of the standard deviation**.
- Both approaches tend to **underestimate the true standard deviation**, but are still used in practice.
- **standard error of the mean** is very useful in machine learning experiments.
- We estimate generalization error by computing **sample mean of error** on test set

Variance and Standard Error

- we can use the standard error to **compute the probability** that the true expectation falls in any chosen interval.
- For example, the **95% confidence interval** centered on the mean $\hat{\mu}_m$ is
- $(\hat{\mu}_m - 1.96SE(\hat{\mu}_m), \hat{\mu}_m + 1.96SE(\hat{\mu}_m))$,
- under the normal distribution with mean $\hat{\mu}_m$ and variance $SE(\hat{\mu}_m)^2$



Variance and Standard Error

- **Bernoulli Distribution**

- $\widehat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$

- $\text{Var}(\widehat{\theta}_m) = \text{Var}\left(\frac{1}{m} \sum_{i=1}^m x^{(i)}\right)$

- $= \frac{1}{m^2} \sum_{i=1}^m \text{Var}(x^{(i)})$

- $= \frac{1}{m^2} \sum_{i=1}^m \theta(1 - \theta)$

- $= \frac{1}{m^2} m \theta(1 - \theta)$

- $= \frac{1}{m} \theta(1 - \theta)$

- variance of estimator decreases as a function of m , number of examples in dataset

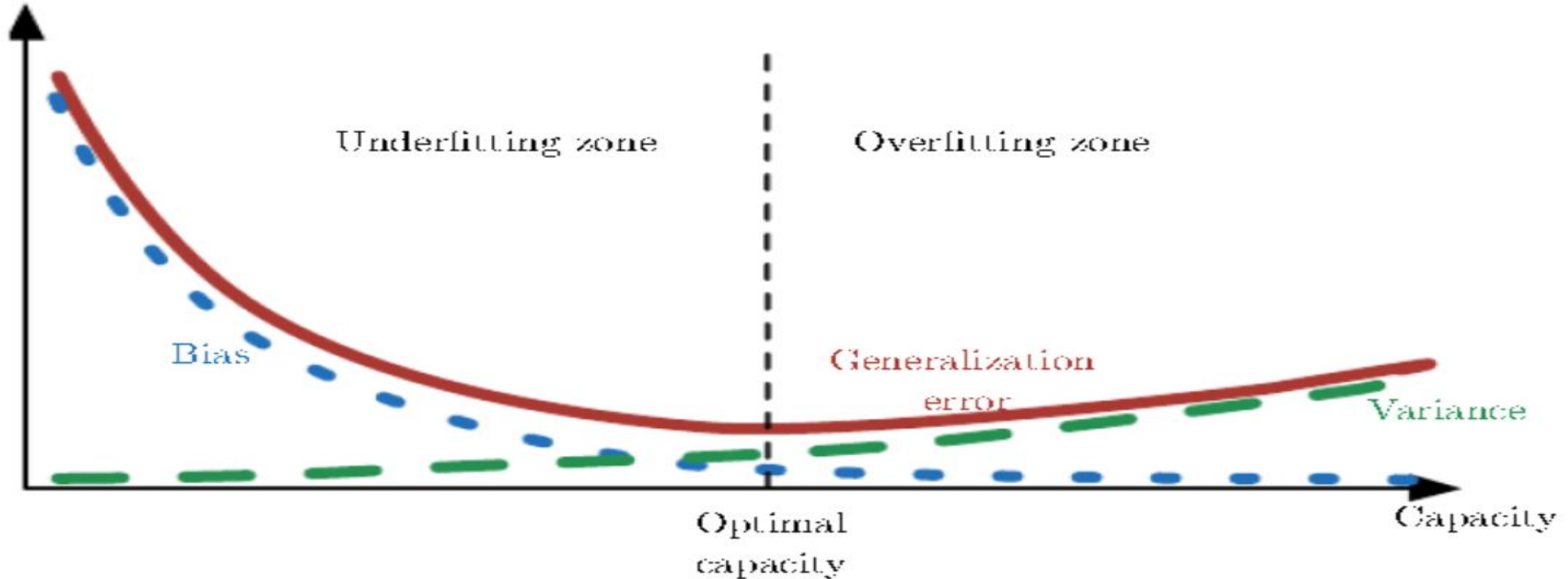
Trading off Bias and Variance to Minimize MSE

- **Bias** is defined as the inability of the model because of that there is **some difference or error** occurring between the **model's predicted value and the actual value**.
- **Bias and variance** measure two different **sources of error in an estimator**.
- Bias measures expected **deviation from the true value** of the function or parameter.
- Variance provides a **measure of the deviation from the expected estimator** value that any particular sampling of the data is likely to cause.
- What happens when we are given a choice between two estimators, **one with more bias and one with more variance?**
- How do we choose between them?
- The most common way to negotiate this trade-off is to use **cross-validation**.

Trading off Bias and Variance to Minimize Mean Squared Error

- we can also compare the mean squared error (MSE) of the estimates.
- $MSE = E [(\widehat{\theta}_m - \theta)]^2$
- $= Bias(\widehat{\theta}_m)^2 + Var(\widehat{\theta}_m)$
- evaluating the MSE incorporates both **bias and the variance**.
- Desirable estimators are those with **small MSE**.
- and these are estimators that manage to keep both their bias and variance somewhat in check

Trading off Bias and Variance to MSE error

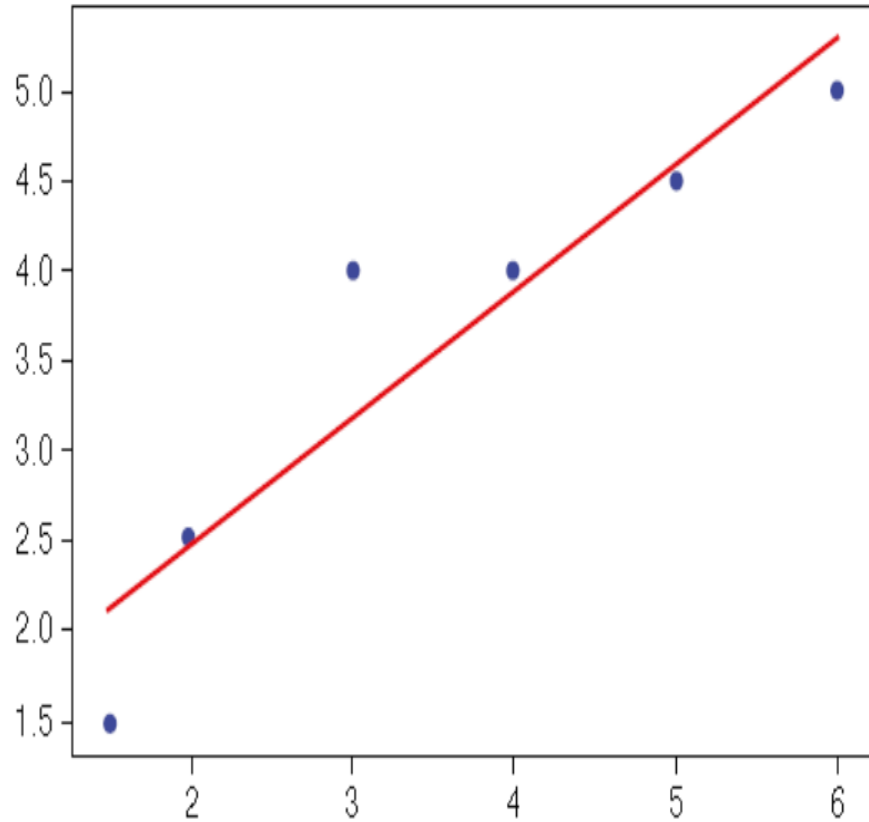


- As capacity increases (x-axis), **bias (dotted) tends to decrease** and **variance (dashed) tends to increase**, yielding another U-shaped curve for generalization error (bold curve).
- If we vary capacity along one axis, there is an optimal capacity, with underfitting when the capacity is below this optimum and overfitting when it is above.

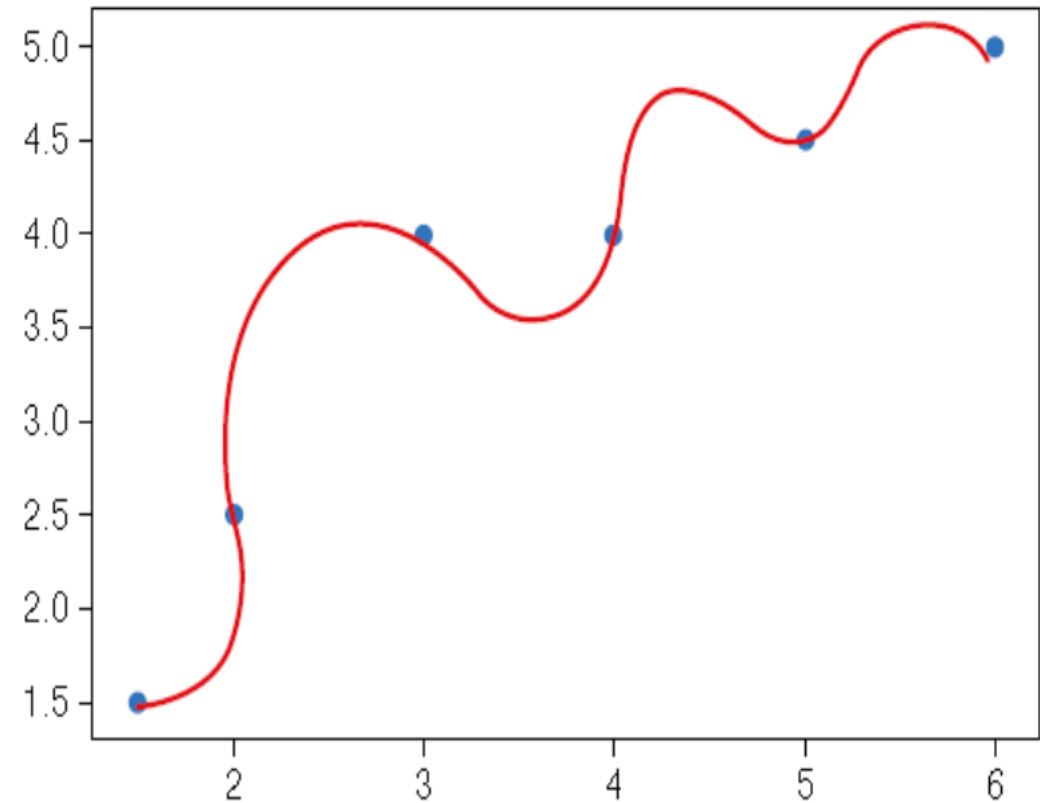
Trading off Bias and Variance to Minimize Mean Squared Error

- The relationship between **bias and variance** is tightly linked to the machine learning concepts of **capacity, underfitting and overfitting**.
- In the case where generalization error is **measured by the MSE**
- (**where bias and variance** are meaningful components of **generalization error**),
- increasing capacity tends **to increase variance and decrease bias**.
- This is illustrated in figure, where we see again the U-shaped curve of generalization error as a function of capacity.

Trading off Bias and Variance to Minimize MSE

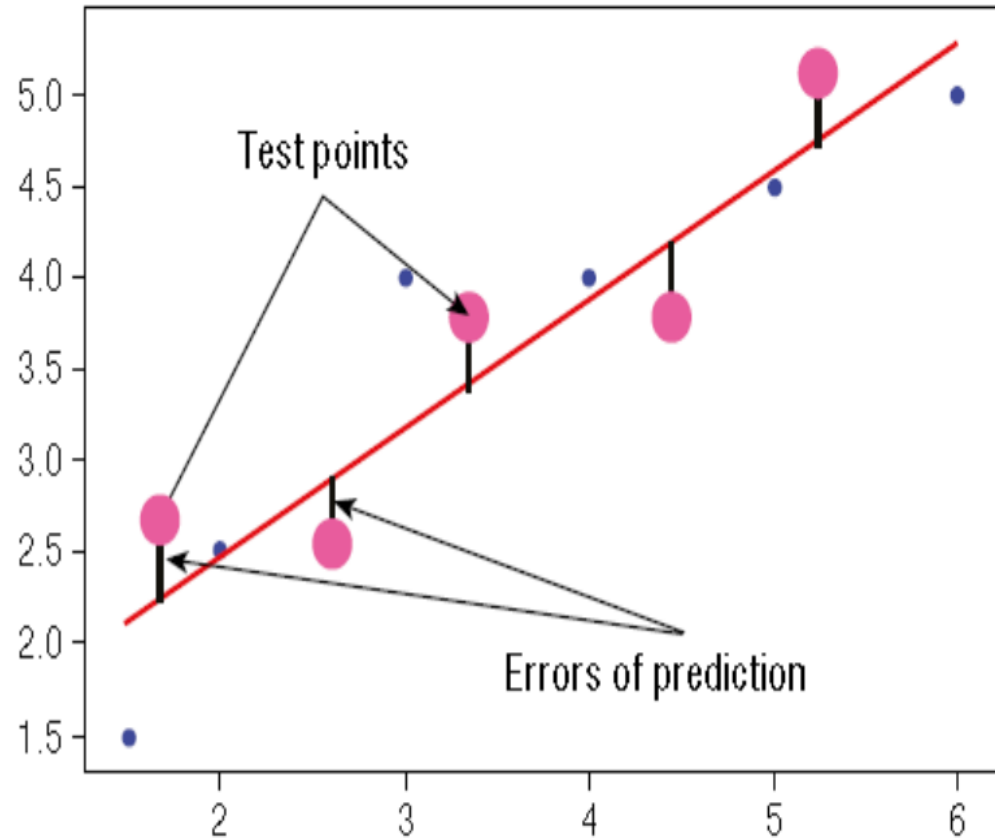


**line can't fit all of the points, so
bias is high**

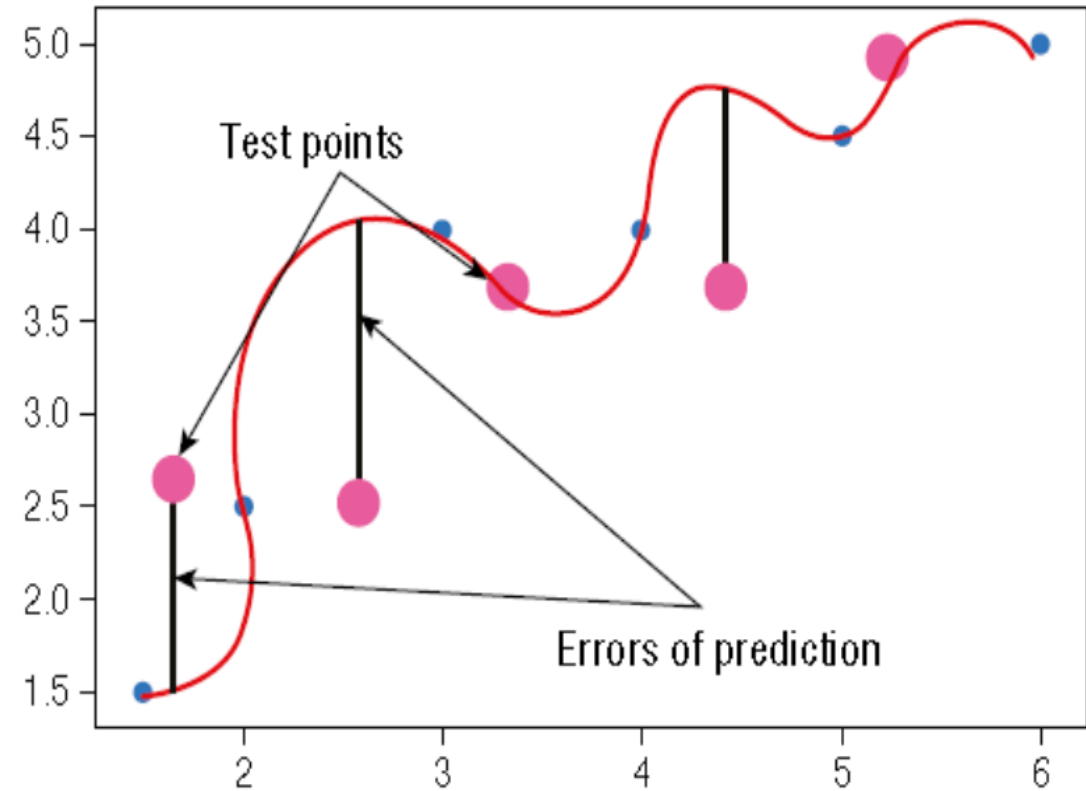


**curvy line fits all of the points, so
bias is low**

Trading off Bias and Variance to Minimize MSE



straight line works well with unseen data, and its result does not vary much with different datasets. Hence, it has low variance.



curvy line does not work well with unseen data, and its result varies with different datasets. Hence, it has high variance

- In machine learning, when we try to **find a curve** that tries to fit all of the points perfectly, it is known as ***overfitting***.
- On the other hand, if we have a line that does not fit most points, it is known as ***underfitting***.
- the ideal algorithm should have the following:
- ***High bias***, with the line hugging as many points as possible
- ***Low variance***, with the line resulting in consistent predictions using different datasets

Consistency

- as number of data points **m in our dataset increases**, our point estimates **converge to true value** of corresponding parameters.
- $\text{Plim } m \rightarrow \infty \widehat{\theta}_m = \theta$.
- symbol plim indicates convergence in probability, for any $\epsilon > 0$,
- $P(|\widehat{\theta}_m - \theta| > \epsilon) \rightarrow 0$ as $m \rightarrow \infty$.
- The condition described by equation is known as **consistency**.
- Almost sure convergence of a sequence of random variables
- $x^{(1)}, x^{(2)}, \dots$ to a **value x occurs** when $p(\lim_{m \rightarrow \infty} x^{(m)} = x) = 1$.
- Consistency ensures that the **bias induced by the estimator diminishes as the number of data examples grows**.

Maximum Likelihood Estimation

- Consider a set of m examples $X = \{x^{(1)}, \dots, x^{(m)}\}$ drawn independently from the true but unknown data generating distribution $p_{\text{data}}(x)$.
- Let $p_{\text{model}}(x; \theta)$ probability distributions over same space indexed by θ .
- $p_{\text{model}}(x; \theta)$ maps any configuration x to a real number estimating the true probability $p_{\text{data}}(x)$.
- The maximum likelihood estimator for θ is then defined as
- $\theta_{ML} = \arg \max_{\theta} p_{\text{model}}(X; \theta)$
- $\quad = \arg \max_{\theta} \prod_{i=1}^m p_{\text{model}}(x^{(i)}; \theta)$

Maximum Likelihood Estimation

- $= \mathit{arg}_{\theta}^{max} \prod_{i=1}^m p_{model}(x^{(i)}; \theta)$
- taking the logarithm of the likelihood does not change its arg max but does conveniently transform a product into a sum:
- $\theta_{ML} = \mathit{arg}_{\theta}^{max} \sum_{i=1}^m \log p_{model}(x^{(i)}; \theta)$
- Because arg max does not change when we rescale the cost function,
- **expectation with respect to empirical distribution $\hat{p}_{data}$ defined by training data:**
- $\theta_{ML} = \mathit{arg}_{\theta}^{max} E_{x \sim \hat{p}_{data}} \log p_{model}(x^{(i)}; \theta)$

Maximum Likelihood Estimation

- One way to interpret MLE is to view it as **minimizing the dissimilarity between empirical distribution $\hat{p}_{data}$ defined by training set and model distribution,**
- with the degree of dissimilarity between the two measured by the **KL divergence.**
- $D_{KL}(\hat{p}_{data} || p_{model}) = E_{x \sim \hat{p}_{data}} [\log \hat{p}_{data}(x) - \log p_{model}(x)]$
- The term on the left is a **function only of the data generating process**, not the model.
- when we train model to **minimize KL divergence**, we need only minimize.

$$-E_{x \sim \hat{p}_{data}} [\log p_{model}(x)]$$

- Minimizing this KL divergence corresponds exactly to minimizing the **crossentropy** between the distributions.

Maximum Likelihood Estimation

- $-E_{x \sim \hat{p}_{data}} [\log p_{model}(x)]$
- Many authors use the term “**cross-entropy**” to identify specifically the **negative log-likelihood of a Bernoulli or softmax distribution**.

For example, **mean squared error** is the **cross-entropy** between the **empirical distribution** and a **Gaussian model**.

- **Maximum likelihood becomes minimization of the negative log-likelihood (NLL), or equivalently, minimization of the cross entropy.**
- KL divergence has a known minimum value of zero.

Conditional Log-Likelihood and Mean Squared Error

- If X represents all our inputs and Y all our observed targets, then the conditional maximum likelihood estimator is
- $\theta_{ML} = \mathit{arg}_{\theta}^{max} P(Y|X; \theta)$
- If the examples are assumed to be i.i.d.,
- then this can be decomposed into
- $\theta_{ML} = \mathit{arg}_{\theta}^{max} \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}; \theta)$

Ex: Linear Regression as Maximum Likelihood

- To derive the same **linear regression algorithm** we obtained before,
- we define $p(\mathbf{y} \mid \mathbf{x}) = N(\mathbf{y}; \hat{\mathbf{y}}(\mathbf{x}; \mathbf{w}), \sigma^2)$.
- function $\hat{\mathbf{y}}(\mathbf{x}; \mathbf{w})$ gives the **prediction of mean of the Gaussian**.
- we assume **variance is fixed** to some **constant σ^2** chosen by the user.
- $\sum_{i=1}^m \log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) =$

$$= -m \log \sigma - \frac{m}{2} \log(2\pi) - \sum_{i=1}^m \frac{\|\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}\|^2}{2\sigma^2},$$

Ex: Linear Regression as Maximum Likelihood

$$= -m \log \sigma - \frac{m}{2} \log(2\pi) - \sum_{i=1}^m \frac{\|\hat{y}^{(i)} - y^{(i)}\|^2}{2\sigma^2},$$

- with the mean squared error,
- $\text{MSE}_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$
- **maximizing the log-likelihood with respect to w yields the same estimate of the parameters w as does minimizing the mean squared error**

Properties of Maximum Likelihood

- MLE can be shown to be the **best estimator asymptotically**, as the number of training **examples $m \rightarrow \infty$** ,
- MLE of a parameter converges to true value of parameter.
- parametric **mean squared error decreases as m increases**,
- maximum likelihood is considered the **preferred estimator** to use for machine learning.

When the **number of examples is small enough** to yield **overfitting behavior**, **regularization strategies such as weight decay** may be used to obtain a biased version of maximum likelihood that **has less variance when training data is limited**.

Bayesian Statistics

- Prior probability distribution, $p(\theta)$ (“the prior”).
- machine learning practitioner selects a prior distribution (i.e. with high entropy) to reflect a high degree of uncertainty in value of θ before observing any data.
- First, unlike the maximum likelihood approach that makes predictions using a point estimate of ϑ , the Bayesian approach is to make predictions using a full distribution over ϑ .
- Now consider that we have a set of data samples $\{x^{(1)}, \dots \dots x^{(m)}\}$
- $p(x^{(1)}, \dots \dots x^{(m)} \mid \theta)$ with the prior via Bayes’ rule:
- $$p(\theta \mid x^{(1)}, \dots \dots x^{(m)}) = \frac{p(x^{(1)}, \dots \dots x^{(m)} \mid \theta) P(\theta)}{p(x^{(1)}, \dots \dots x^{(m)})}$$

Bayesian Statistics

- after observing m examples, the predicted distribution over the next data sample, $x^{(m+1)}$, is given by
- $p(x^{(m+1)} \mid x^{(1)}, \dots, x^{(m)}) = \int p(x^{(m+1)} \mid \theta) p(\theta \mid x^{(1)}, \dots, x^{(m)}) d\theta$
- The **variance** of the estimator is an assessment of how the estimate might change **with alternative samplings** of the observed data.
- how to deal with the **uncertainty in the estimator** is to simply **integrate over it, which tends to protect well against overfitting**.
- Bayesian methods **generalize much better when limited training data is available**,
- but typically suffer from **high computational cost when the number of training examples is large**.

Example: Bayesian Linear Regression

- In linear regression, we learn a linear mapping from an input vector $x \in \mathbf{R}^n$ to predict the value of a scalar $y \in R$.
- The prediction is parametrized by the vector $w \in \mathbf{R}^n$:
- $\hat{y} = w^T x$
- Given a set of m training samples $(X^{(train)}, Y^{(train)})$, we can express the prediction of y over the entire training set as:
- $\hat{y}^{(train)} = X^{(train)} w$

Example: Bayesian Linear Regression

- Expressed as a Gaussian conditional distribution on $\mathbf{Y}^{(train)}$, we have
- $\hat{\mathbf{y}}^{(train)} = \mathbf{X}^{(train)} \mathbf{w}$
- $p(\mathbf{y}^{(train)} \mid \mathbf{X}^{(train)}, \mathbf{w}) = \mathcal{N}(\mathbf{y}^{(train)}; \mathbf{X}^{(train)} \mathbf{w}, \mathbf{I})$
- $\propto \exp\left(-\frac{1}{2} (\mathbf{y}^{(train)} - \mathbf{X}^{(train)} \mathbf{w})^t (\mathbf{y}^{(train)} - \mathbf{X}^{(train)} \mathbf{w})\right)$
- assuming that the **Gaussian variance on $\mathbf{y}$ is one**.
- **we refer to $(\mathbf{X}^{(train)}, \mathbf{y}^{(train)})$ as simply $(\mathbf{X}, \mathbf{y})$.**
- $P(\mathbf{w}) = \mathcal{N}(\mathbf{w}; \mu_0, \Lambda_0) \propto \exp\left(-\frac{1}{2} (\mathbf{w} - \mu_0)^t \Lambda_0^{-1} (\mathbf{w} - \mu_0)\right)$
- where μ_0 and Λ_0 are the prior distribution mean vector and covariance matrix respectively

Example: Bayesian Linear Regression

- Posterior distribution over the model parameters.
- $P(w|X, y) \propto p(y|X, w)p(w)$
$$\propto \exp\left(-\frac{1}{2} (y - Xw)^t (y - Xw)\right) \exp\left(-\frac{1}{2} (w - \mu_0)^t \Lambda_0^{-1} (w - \mu_0)\right)$$
$$\propto \exp\left(-\frac{1}{2} \left(-2y^t Xw + w^t X^t Xw + w^t \Lambda_0^{-1} w - 2\mu_0^t \Lambda_0^{-1} w\right)\right)$$
-

Example: Bayesian Linear Regression

- We now define , $\Lambda_m = (X^t X + \Lambda_0^{-1})^{-1}$
- And $\mu_m = \Lambda_m (X^t y + \Lambda_0^{-1} \mu_0)$
- $$P(w|X, y) \propto \exp \left(-\frac{1}{2} (w - \mu_m)^T \Lambda_m^{-1} (w - \mu_m) \right) + \frac{1}{2} \mu_m^t \Lambda_m^{-1} \mu_m$$
$$\propto \exp \left(-\frac{1}{2} (w - \mu_m)^T \Lambda_m^{-1} (w - \mu_m) \right)$$
- All terms that do not include the parameter vector w have been omitted;
- they are implied by fact that **distribution must be normalized to integrate to 1.**
- Bayesian estimate provides a covariance matrix, showing how likely all the different values of w are, rather than providing only the estimate μ_m .

Maximum *A Posteriori* (MAP) Estimation

- The MAP estimate chooses the point of maximal posterior probability.
- $\theta_{MAP} = \mathit{arg}_{\theta}^{max} P(\theta|x) = \mathit{arg}_{\theta}^{max} \log p(x| \theta) + \log p(\theta)$
- $\log p(x / \theta)$, i.e. the **standard loglikelihood**
- and $\log p(\theta)$, corresponding to the **prior distribution**
- It has the advantage of leveraging information that is brought by the **prior** and cannot be found in the training data.
- This additional information **helps to reduce the variance** in the MAP point estimate (in comparison to the ML estimate).

Maximum *A Posteriori* (MAP) Estimation

- Many regularized estimation strategies, such as maximum likelihood learning **regularized with weight decay**, can be interpreted as making the **MAP approximation to Bayesian inference**.
- This view applies when the **regularization** consists of adding an extra term to the objective function that corresponds to **$\log p(\theta)$** .
- For example, a **more complicated penalty term can be derived by** using a **mixture of Gaussians**, rather than a single Gaussian distribution, as the prior

Supervised Learning Algorithms

- learning algorithms that learn to **associate some input with some output**,
- given a training set of examples of **inputs x and outputs y** .

Probabilistic Supervised Learning

- **linear regression**
- $p(y|x; \theta) = \mathbf{N}(y; \theta^t x, I)$
- If we have two classes, class 0 and class 1, then we need only specify the probability of one of these classes.
- The probability of class 1 determines the probability of class 0, because these two values must add up to 1.

Supervised Learning Algorithms

- **normal distribution** over real-valued numbers that we used for linear regression is parametrized in terms of a **mean**.

A distribution over a binary variable is slightly more complicated, because its **mean must always be between 0 and 1**.

- One way to solve this problem is to use the **logistic sigmoid function** to squash the output of the linear function into the interval (0, 1) and interpret that value as a probability:
- $p(y=1|x; \theta) = \sigma(\theta^t x)$
- This approach is known as **logistic regression**

Supervised Learning Algorithms

- In the case of **linear regression**, able to **find the optimal weights** by solving the normal equations.
- **Logistic regression is somewhat more difficult.**
- There is **no** closed-form solution **for its optimal weights**.
- we must search for them by **maximizing the log-likelihood**.
- We can do this by **minimizing the negative log-likelihood (NLL) using gradient descent**.

Support Vector Machines

- supervised learning is the **support vector machine**.
- similar to **logistic regression** , it is driven by a linear function $\mathbf{w}^t \mathbf{x} + b$.
- Unlike logistic regression, the support vector machine does not provide probabilities, but only **outputs a class identity**.
- SVM predicts that **positive class** is present when $\mathbf{w}^t \mathbf{x} + b$ is **positive**.
- Likewise, it predicts negative class is present when $\mathbf{w}^t \mathbf{x} + b$ is **negative**.
- One key innovation associated with SVM machines is **kernel trick**.
- kernel trick can be written in terms of **dot products**.

Support Vector Machines

- For example,
- linear function used by support vector machine can be re-written as
- $\mathbf{w}^t x + b = \mathbf{b} + \sum_{i=1}^m \alpha_i x^t x^{(i)}$
- where $x^{(i)}$ is a training example and α is a vector of coefficients.
- replace x by output of given feature function $\varphi(x)$
- and dot product with a function
- $k(x, x^{(i)}) = \varphi(x) \cdot \varphi(x^{(i)})$ called a kernel.
- After replacing dot products with kernel evaluations, we can make predictions
- $\mathbf{b} + \sum_{i=1}^m \alpha_i k(x, x^{(i)})$

Support Vector Machines

- This function is nonlinear with respect to x ,
- but the relationship between $\varphi(x)$ and $f(x)$ is linear.
- Also, the relationship between α and $f(x)$ is linear.
- The kernel-based function is exactly equivalent to **preprocessing the data**
- by applying $\varphi(x)$ to all inputs, then learning a linear model in the new transformed space

Support Vector Machines

- **The kernel trick is powerful for two reasons.**
- First, it allows us to learn models **that are nonlinear** as a function of x using convex optimization techniques that are guaranteed to converge efficiently.
- This is possible because we consider φ fixed and optimize only α ,
- i.e., the optimization algorithm **can view the decision function as being linear in a different space.**

Second, **more computational efficient** than naively constructing two vectors and explicitly taking $\varphi(x)$ their dot product.

The most commonly used kernel is the **Gaussian kernel**.

Support Vector Machines

- SVM are not only algorithm that can be enhanced using kernel trick.
- Many other linear models can be enhanced in this way.
- **A major drawback to kernel machines** is that the cost of evaluating the decision function is linear in the number of training examples, because the i -th example contributes a term $\alpha_i k(x, x^{(i)})$ to the decision function.
- SVM are able to mitigate this by learning an α vector that contains mostly zeros.
- Classifying a new example then requires evaluating the **kernel function only for the training examples that have non-zero α_i** .
- Kernel machines also suffer from a **high computational cost of training when the dataset is large**

Other Simple Supervised Learning Algorithms

- **k -nearest neighbors is a family of techniques** that can be used for classification or regression.
- **As a non-parametric learning algorithm,**
- there is not even really a training stage or learning process.
- at test time, when we want to produce an output y for a new test input x ,
- we find the k -nearest neighbors to x in the training data X .

Other Simple Supervised Learning Algorithms

- This works for any kind of supervised learning where we can define an average over y values.
- In the case of classification, we can average over **one-hot code** vectors c with $c_y = 1$ and $c_i = 0$ for all other values of i .
- We can then interpret the average over these one-hot codes as giving a probability distribution over classes.
- As a non-parametric learning algorithm, k -nearest neighbor can achieve very high capacity

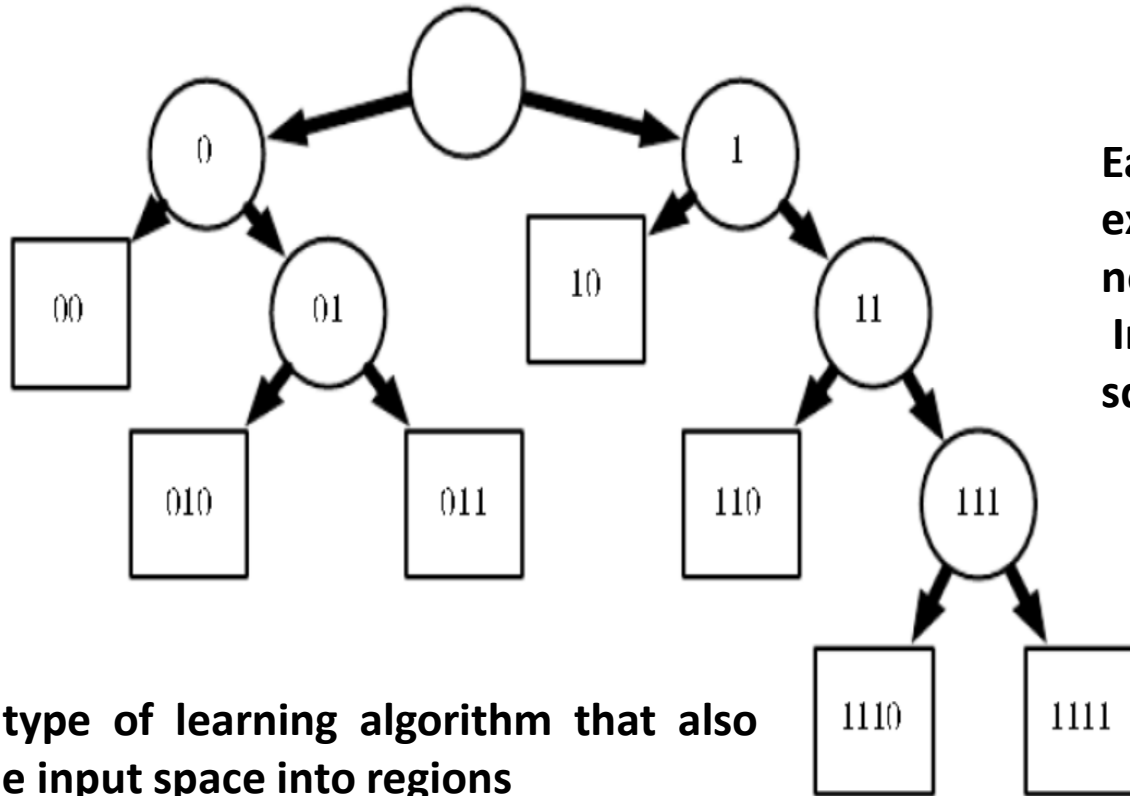
Other Simple Supervised Learning Algorithms

- When there is infinite training data,
- all test points x will have infinitely many training set neighbors at distance zero.
- If we allow the **algorithm to use** all of these neighbors to vote, the procedure converges to the **Bayes error rate**.
- The **high capacity of k -nearest** neighbors allows it to obtain high accuracy given a large training set.
- However, it does so at high computational cost, and it may generalize very badly given a small, finite training set.

Other Simple Supervised Learning Algorithms

- One weakness of k -nearest neighbors is that **it cannot learn that one feature is more discriminative than another.**
- The nearest neighbor of most points x will be determined by the large number of features x_2 through x_{100} , not by the alone feature x_1 .
- Thus the output on small training sets will essentially be random.

Decision tree

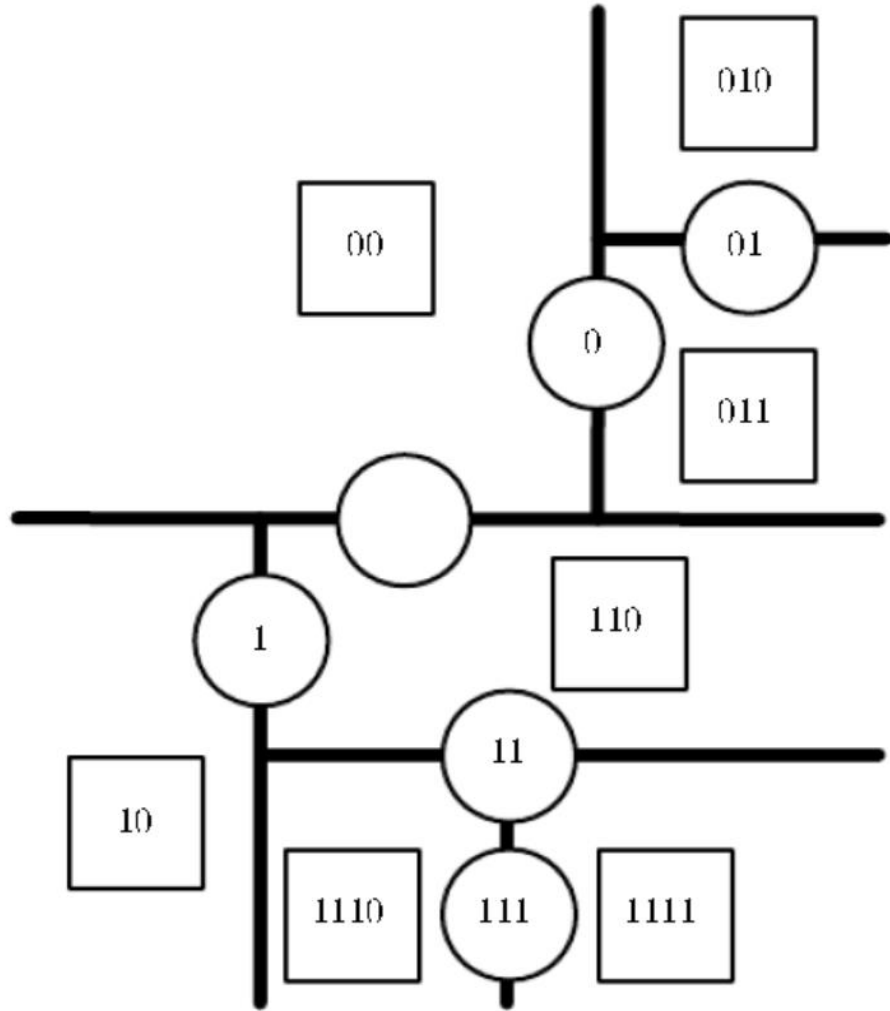


Each node of the tree chooses to send the input example to the child node on the left (0) or to the child node on the right (1).

Internal nodes are drawn as circles and leaf nodes as squares

Another type of learning algorithm that also breaks the input space into regions and has separate parameters for each region is the decision tree

Decision tree



The tree divides space into regions
with each internal node drawn along the
dividing line it uses to categorize examples,
and leaf nodes drawn in the center of the
region of examples they receive

Decision tree

- each node of the decision tree is associated with a region in the input space,
- internal nodes break that region into one sub-region for each child of the node.
- Space is thus sub-divided into non-overlapping regions, with a one-to-one correspondence between leaf nodes and input regions.
- Each leaf node usually maps every point in its input region to the same output.
- Decision trees are usually trained with specialized algorithms.
- The learning algorithm can be considered **non-parametric** if it is allowed to learn a tree of arbitrary size,
- decision trees are usually **regularized with size constraints** that turn them into **parametric models in practice**.

Decision tree

- Decision trees as they are typically used, with axis-aligned splits and constant outputs within each node, struggle to solve some problems that are easy even for **logistic regression**.
- For example, if we have a two-class problem and the positive class occurs wherever $x_2 > x_1$, the decision boundary is not axis-aligned

Unsupervised Learning Algorithms

- unsupervised algorithms are those that experience only “features”
but not a supervision signal.
- there is no objective test for distinguishing whether a value is a feature or a target provided by a supervisor.
- attempts to extract information from a distribution that do not require human labor to annotate examples.
- associated with density estimation, learning to draw samples from a distribution, learning to denoise data from some distribution,
- clustering the data into groups of related examples

Unsupervised Learning Algorithms

- to find the “best” representation of the data.
- Looking for a representation that preserves much information about x .
- There are multiple ways of defining a *simpler* representation.
- Three of the most common include **lower dimensional representations, sparse representations and independent representations**.
- Low-dimensional representations attempt to compress as much information about x as possible in a smaller representation.
- Sparse representations embed the dataset into a representation whose entries are **mostly zeroes for most inputs**

Unsupervised Learning Algorithms

- The use of **sparse representations** requires increasing the dimensionality of the representation,
- so that representation becoming mostly zeroes does not discard too much information.
- This results in an overall structure of the representation that tends to distribute data along the axes of the representation space.
- **Independent representations**
- the dimensions of the representation are statistically independent.

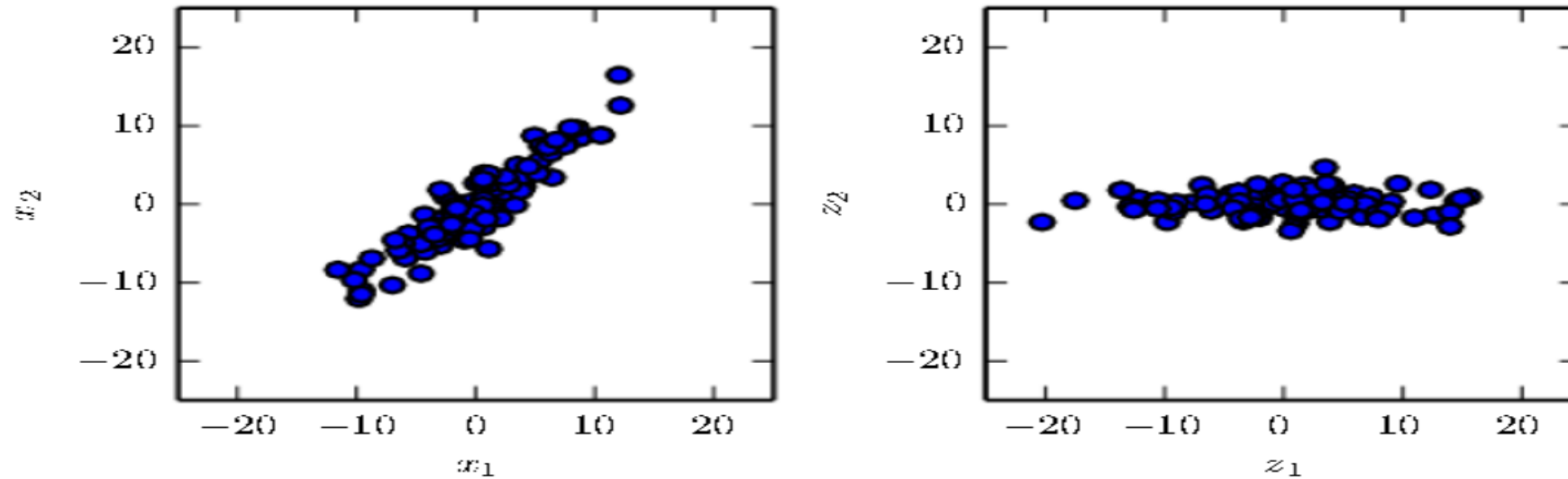
Unsupervised Learning Algorithms

- Low dimensional representations yield elements that have fewer or weaker dependencies than the original high-dimensional data.
- Identifying and removing more redundancy allows the dimensionality reduction algorithm
- to achieve more compression while discarding less information.

Principal Components Analysis (PCA)

- PCA algorithm provides a means of compressing data.
- We can also view **PCA as an unsupervised learning algorithm** that learns a representation of data.
- PCA learns a representation that **has lower dimensionality** than the original input.
- It also learns a representation whose elements **have no linear correlation with each other**.
- This is a first step toward the criterion of learning representations whose elements are statistically independent.

Principal Components Analysis (PCA)



- PCA learns a **linear projection that aligns direction of greatest variance** with axes of new space.
- *(Left)* The original data consists of samples of x .
In this space, the variance might occur along directions that are not axis-aligned.
- *(Right)* The transformed data $z = x^t W$ now varies most along the axis z_1 .
- The direction of second most variance is now along z_2

Principal Components Analysis (PCA)

- PCA learns an orthogonal, linear transformation of the data that projects an input x to a representation z .
- we can use PCA as a simple and **effective dimensionality reduction** method that preserves as much of the information in the data as possible (measured by least-squares reconstruction error).
- Let us consider the $m \times n$ -dimensional design matrix X .
- We will assume that the data has a mean of zero, $E[x] = 0$.
- If this is not the case, the data can easily be centered by subtracting the mean from all examples in a preprocessing step

Principal Components Analysis (PCA)

- The **unbiased sample covariance** matrix associated with X is given by:
- $\text{Var} [x] = \frac{1}{m-1} X^t X$
- PCA finds a representation (through linear transformation) $z = x^t W$
- where $\text{Var}[z]$ is diagonal.
- principal components of a design matrix X ,
- are given by the eigenvectors of $X^t X$.
- $X^t X = W \Lambda W^t$

Principal Components Analysis (PCA)

- Principal components may also be obtained via singular value decomposition.
- they are the right singular vectors of X .
- let W be the right singular vectors in the decomposition $X = U\Sigma W^t$. We then recover the original eigenvector equation with W as the eigenvector basis:
- $X^t X = (U\Sigma W^t)^t U\Sigma W^t = W \Sigma^2 W^t$
- The SVD is helpful to show that PCA results in a diagonal Var $[z]$.

Principal Components Analysis (PCA)

- Using the SVD of X , we can express the variance of X as:
- $\text{Var} [x] = \frac{1}{m-1} X^t X$
- $\frac{1}{m-1} (U \Sigma W^t)^t U \Sigma W^t$
- $\frac{1}{m-1} W \Sigma^T U^T U \Sigma W^t$
- $= \frac{1}{m-1} W \Sigma^2 W^t$
- Since $U^T U = I$ because the U matrix of the singular value decomposition is defined to be orthogonal

Principal Components Analysis (PCA)

- This shows that if we take $z = \mathbf{x}^t \mathbf{W}$,
- we can ensure that the covariance of z is diagonal as required:
- $\text{Var} [z] = \frac{1}{m-1} \mathbf{Z}^t \mathbf{Z}$
- $\frac{1}{m-1} \mathbf{W}^t \mathbf{X}^t \mathbf{X} \mathbf{W}$
- $\frac{1}{m-1} \mathbf{W}^t \mathbf{W} \Sigma^2 \mathbf{W}^t \mathbf{W}$
- $= \frac{1}{m-1} \Sigma^2$
- Since $\mathbf{W}^T \mathbf{W} = \mathbf{I}$ because singular value decomposition is defined to be orthogonal

Principal Components Analysis (PCA)

- $\text{Var}(\mathbf{z}) = \frac{1}{m-1} \Sigma^2$
- The above analysis shows that when we project **the data $\mathbf{x}$ to $\mathbf{z}$** , via the linear transformation W ,
- the resulting representation has a diagonal covariance matrix (as given by Σ^2)
- which implies **that individual elements of $\mathbf{z}$ are mutually uncorrelated**.
- This ability of PCA to transform data into a representation where the elements are **mutually uncorrelated is a very important property of PCA**.
- It is a simple example of a representation that attempts to *disentangle the unknown factors of variation* underlying the data.

Principal Components Analysis (PCA)

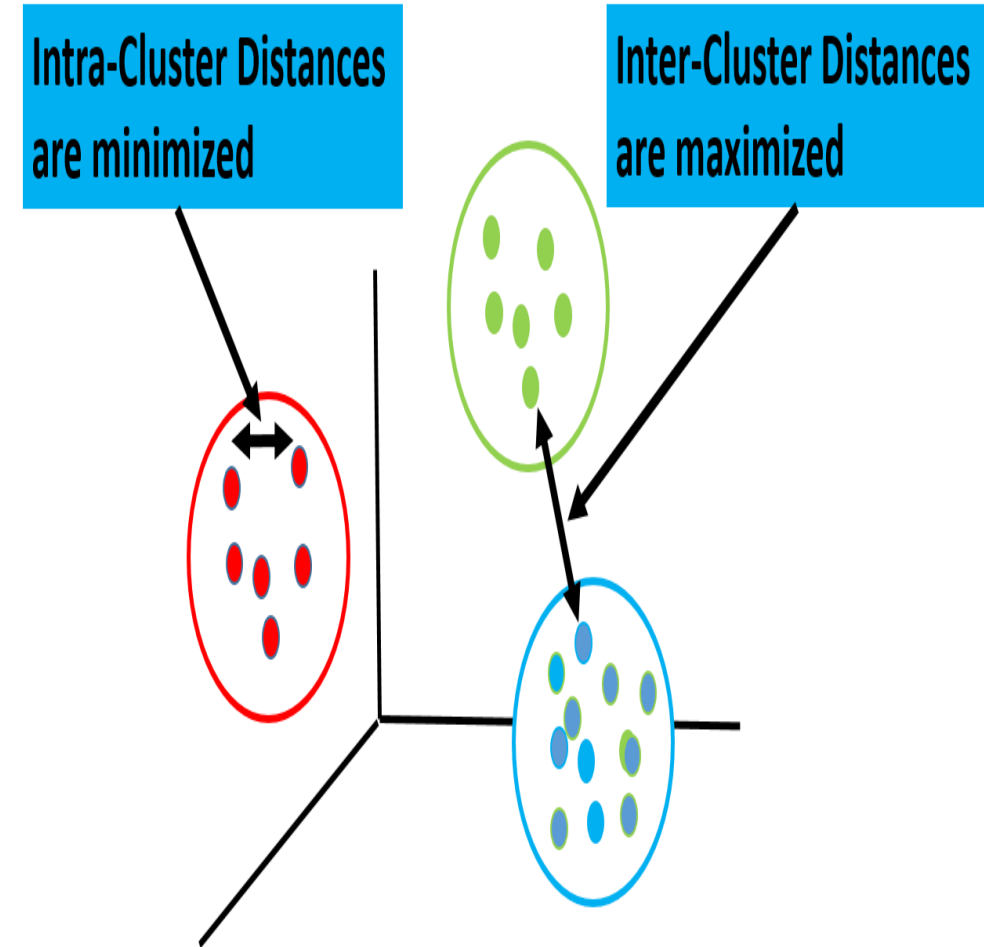
- In the case of PCA, this disentangling takes the form of finding a rotation of the input space (described by W) that aligns the principal axes of variance with the basis of the new representation space associated with z .
- While **correlation is an important category of dependency between elements of the data**

k -means Clustering

- The k -means clustering algorithm divides the training set into k different clusters of examples that are near each other.
- We can thus think of the algorithm as providing a k -dimensional one-hot code vector h representing an input x .
- If x belongs to cluster i , then $h_i = 1$ and all other entries of the representation h are zero.
- **The one-hot code provided by k -means clustering is an example of a sparse representation, because the majority of its entries are zero for every input.**

Slides for teaching presentation-Clustering

- **Cluster:** a collection of data objects
 - Similar to one another within the same cluster
 - Dissimilar to the objects in other clusters
- **Cluster analysis**
 - Grouping a set of data objects into clusters
- Clustering is **unsupervised classification**: no predefined classes.
- A good clustering method will produce high quality clusters with
 - high intra-class similarity
 - low inter-class similarity



Clustering

Attributes/ Feature -2

Data sample or instance records -4

What will be the output?

Requirement: How many clusters do you want?

C= 2

Output may look like may belong to either 1 or 2 cluster

C1 = D1, D3

C2 = D2, D4

	Attribute 1	Attribute 2
D1	1	1
D2	2	1
D3	4	3
D4	5	4

	Cluster
D1	1
D2	2
D3	1
D4	1

K-Mean Clustering

	Attribute 1	Attribute 2
D1	1	1
D2	2	1
D3	4	3
D4	5	4

So based on Logic True false from Distance Matrix (Min value is 1)		
	Centroid1	Centroid 2
D1(1,1)	1	0
D2(2,1)	1	0
D3(4,3)	0	1
D4(5,4)	1	0

C= 2, Take Random Centroids

Centroid 1		Centroid 2	
1	1	4	3

Build Distance Matrix (Euclidean Distance)		
	Centroid1 (1,1)	Centroid 2 (4,3)
D1(1,1)	0	0.1
D2(2,1)	2	5
D3(4,3)	6	5
D4(5,4)	8	9

New Centroid Find Mean value :

C1 : D1, D2, D4 ,

so C1 New= $(1+2+5)/3, (1+1+4)/3 = (2.67, 2)$

C2: D3 , So C2 New = (4, 3)

Stochastic Gradient Descent (SGD)

- Nearly all of deep learning is powered by one very important algorithm: stochastic gradient descent or SGD.
- A recurring problem in machine learning is that **large training** sets are necessary for **good generalization**,
- but large training sets are also more **computationally expensive**.
- The cost function used by a machine learning algorithm often decomposes as a sum over training examples of some per-example loss function.

Stochastic Gradient Descent (SGD)

- For example, the negative conditional log-likelihood of the training data can be written as

- $J(\theta) = E_{x \sim \hat{p}_{data}} L(x, y, \theta) = \frac{1}{m} \sum_{i=1}^m L(x^{(i)}, y^{(i)}, \theta)$

- *where* L is the per-example loss $L(x^{(i)}, y^{(i)}, \theta) = -\log P(y|x; \theta)$

- For these additive cost functions, **gradient descent requires computing**

- $\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(x^{(i)}, y^{(i)}, \theta)$

computational cost of this operation is $O(m)$.

As the training set size grows to billions of examples, the time to take a single gradient step becomes prohibitively long.

Stochastic Gradient Descent (SGD)

- The insight of stochastic gradient descent is that the gradient is an expectation.
- The expectation may be approximately estimated using a small set of samples.
- on each step of the algorithm, we can sample a minibatch of examples
- $B = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ drawn uniformly from the training set.
- The minibatch size m' is typically chosen to be a relatively small number of examples, ranging from 1 to a few hundred.
- We may fit a training set with billions of examples using updates computed on only a hundred examples

Stochastic Gradient Descent (SGD)

- The estimate of the gradient is formed as
- $\mathbf{g} = \frac{1}{m'} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, \boldsymbol{\theta})$
- stochastic gradient descent algorithm then follows the estimated gradient downhill:
- $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \mathbf{g}$
- where ϵ is the learning rate.
- Gradient descent in general has been regarded as slow or unreliable.
- In the past, the application of gradient descent to non-convex optimization problems was regarded as unprincipled

Stochastic Gradient Descent (SGD)

- optimization algorithm may **not be guaranteed to arrive at even a local minimum** in a reasonable amount of time,
- but it often finds a very low value of the cost function quickly enough to be useful.
- **Stochastic gradient descent has many important uses**
- It is the main way to **train large linear models** on very large datasets.
- For a fixed model size, cost per SGD update does not depend on the training set size m .
- we use a larger model as the training set size increases,
- as m approaches infinity, the model will eventually converge to its best possible test error before SGD has sampled every example in the training set

Stochastic Gradient Descent (SGD)

- Prior to the advent of deep learning, the main way to **learn nonlinear models** was to use the **kernel trick** in combination with a linear model.
- Many kernel learning algorithms require constructing an $m \times m$ matrix $G_{i,j} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$

Building a Machine Learning Algorithm

- combine a specification of a **dataset**, a **cost function**, an **optimization procedure** and a **model**.
- For example, the linear regression algorithm combines a dataset consisting of X and y , the cost function
- $J(\mathbf{w}, \mathbf{b}) = -E_{x \sim \hat{p}_{data}} \log p_{model}(y|x)$
- the model specification, $p_{model}(y|x) = N(y; x^t \mathbf{w} + \mathbf{b}, 1)$,
- and, in most cases, the optimization algorithm defined by solving for where the gradient of the cost is zero using the normal equations.

Building a Machine Learning Algorithm

- $J(\mathbf{w}, \mathbf{b}) = -E_{\mathbf{x} \sim \hat{p}_{data}} \log p_{model}(\mathbf{y}|\mathbf{x})$
- the model specification, $p_{model}(\mathbf{y}|\mathbf{x}) = N(\mathbf{y}; \mathbf{x}^t \mathbf{w} + \mathbf{b}, 1)$,
- The most common cost function is the negative log-likelihood,
- so that minimizing the cost function causes maximum likelihood estimation.
- cost function may include additional terms, such as **regularization terms**.
- For example, we can add weight decay to the linear regression cost function
- $J(\mathbf{w}, \mathbf{b}) = \lambda \|\mathbf{w}\|_2^2 - E_{\mathbf{x} \sim \hat{p}_{data}} \log p_{model}(\mathbf{y}|\mathbf{x})$
- This still allows closed-form optimization

Building a Machine Learning Algorithm

- If we change the model to be nonlinear, then most cost functions can no longer be optimized in closed form.
- This requires us to choose an **iterative numerical optimization procedure, such as gradient descent.**
- constructing a learning algorithm by **combining models, costs, and optimization algorithms supports both supervised and unsupervised learning.**
- The linear regression example shows how to support supervised learning.
- Unsupervised learning can be supported by defining a **dataset that contains only X and providing an appropriate unsupervised cost and model.**

Building a Machine Learning Algorithm

- For example, we can obtain the first PCA vector by specifying that our loss function is
- $\mathbf{J}(\mathbf{w}) = E_{\mathbf{x} \sim \hat{p}_{data}} \|\mathbf{x} - \mathbf{r}(\mathbf{x}; \mathbf{w})\|_2^2$
- while our model is defined to have w with norm one and reconstruction function
- $r(\mathbf{x}) = \mathbf{w}^t \mathbf{x} \mathbf{w}$.
- Some models such as decision trees or k -means require special-case optimizers because their cost functions have flat regions that make them inappropriate for minimization by gradient-based optimizers.

Maximum likelihood estimation for simple linear regression

- Given a simple linear regression model with independent observations.
- $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i \sim N(0, \sigma^2), i = 1, \dots, n,$
- the maximum likelihood estimates of β_0 , β_1 , and σ^2 are given by
 - $$\widehat{\beta}_0 = \bar{y} - \widehat{\beta}_1 \bar{x}$$
 - $$\widehat{\beta}_1 = \frac{s_{xy}}{s_x^2}$$
 - $$\widehat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \widehat{\beta}_0 - \widehat{\beta}_1 x_i)^2$$
- Where $\bar{x}$, $\bar{y}$ are the sample means,
- s_x^2 is sample variance of x
- s_{xy} is the sample covariance between x and y

Maximum likelihood estimation for simple linear regression

- Proof

$$\begin{aligned} p(y|\beta_0, \beta_1, \sigma^2) &= \prod_{i=1}^n p(y_i|\beta_0, \beta_1, \sigma^2) \\ &= \prod_{i=1}^n \mathcal{N}(y_i; \beta_0 + \beta_1 x_i, \sigma^2) \\ &= \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left[-\frac{(y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma^2}\right] \\ &= \frac{1}{\sqrt{(2\pi\sigma^2)^n}} \cdot \exp\left[-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\right] \end{aligned}$$

Maximum likelihood estimation for simple linear regression

- and the log-likelihood function

$$\begin{aligned}\text{LL}(\beta_0, \beta_1, \sigma^2) &= \log p(y|\beta_0, \beta_1, \sigma^2) \\ &= -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 .\end{aligned}$$

Maximum likelihood estimation for simple linear regression

$$\frac{dLL(\beta_0, \beta_1, \sigma^2)}{d\beta_0} = \frac{1}{\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)$$

and setting this derivative to zero gives the MLE for β_0 :

$$\frac{dLL(\hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}^2)}{d\beta_0} = 0$$

$$0 = \frac{1}{\hat{\sigma}^2} \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)$$

$$0 = \sum_{i=1}^n y_i - n\hat{\beta}_0 - \hat{\beta}_1 \sum_{i=1}^n x_i$$

$$\hat{\beta}_0 = \frac{1}{n} \sum_{i=1}^n y_i - \hat{\beta}_1 \frac{1}{n} \sum_{i=1}^n x_i$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} .$$

Maximum likelihood estimation for simple linear regression

The derivative of the log-likelihood function (4) at $\hat{\beta}_0$ with respect to β_1 is

$$\frac{dLL(\hat{\beta}_0, \beta_1, \sigma^2)}{d\beta_1} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i y_i - \hat{\beta}_0 x_i - \beta_1 x_i^2)$$

and setting this derivative to zero gives the MLE for β_1 :

$$\frac{dLL(\hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}^2)}{d\beta_1} = 0$$

$$0 = \frac{1}{\hat{\sigma}^2} \sum_{i=1}^n (x_i y_i - \hat{\beta}_0 x_i - \hat{\beta}_1 x_i^2)$$

$$0 = \sum_{i=1}^n x_i y_i - \hat{\beta}_0 \sum_{i=1}^n x_i - \hat{\beta}_1 \sum_{i=1}^n x_i^2$$

Maximum likelihood estimation for simple linear regression

$$0 \stackrel{(6)}{=} \sum_{i=1}^n x_i y_i - (\bar{y} - \hat{\beta}_1 \bar{x}) \sum_{i=1}^n x_i - \hat{\beta}_1 \sum_{i=1}^n x_i^2$$

$$0 = \sum_{i=1}^n x_i y_i - \bar{y} \sum_{i=1}^n x_i + \hat{\beta}_1 \bar{x} \sum_{i=1}^n x_i - \hat{\beta}_1 \sum_{i=1}^n x_i^2$$

$$0 = \sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} + \hat{\beta}_1 n\bar{x}^2 - \hat{\beta}_1 \sum_{i=1}^n x_i^2$$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n x_i y_i - \sum_{i=1}^n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - \sum_{i=1}^n \bar{x}^2}$$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\hat{\beta}_1 = \frac{s_{xy}}{s_x^2} .$$

Maximum likelihood estimation for simple linear regression

The derivative of the log-likelihood function (4) at $(\hat{\beta}_0, \hat{\beta}_1)$ with respect to σ^2 is

$$\frac{dLL(\hat{\beta}_0, \hat{\beta}_1, \sigma^2)}{d\sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

and setting this derivative to zero gives the MLE for σ^2 :

$$\frac{dLL(\hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}^2)}{d\sigma^2} = 0$$

$$0 = -\frac{n}{2\hat{\sigma}^2} + \frac{1}{2(\hat{\sigma}^2)^2} \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

$$\frac{n}{2\hat{\sigma}^2} = \frac{1}{2(\hat{\sigma}^2)^2} \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2 .$$

References

- Text book:
- 1. Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning (2016) An
- MIT Press book, <http://www.deeplearningbook.org>.(T2)
- 2. Skansi S., Introduction to Deep Learning - From Logical Calculus to Artificial
- Intelligence, 1st Edition, Springer International Publishing, 2018.(T2)
- Reference book:
- Buduma N., Fundamentals of Deep Learning, 1st Edition, O Reilly Media, 2016.(R1)
- **Wei-Meng Lee , Python Machine Learning, Wiley Publishing.**
- <https://statproofbook.github.io/P/slr-mle.html>